Online Job Selection: Reward Rate vs. Remaining Value

Mohammad Reza Aminian
Booth School of Business, University of Chicago, Chicago, Illinois 60637
maminian@chicagobooth.edu

Will Ma
Graduate School of Business, Columbia University, New York 10027
wm2428@gsb.columbia.edu

Linwei Xin
School of Operations Research and Information Engineering, Cornell University, Ithaca, NY 14853
lx267@cornell.edu

High-throughput systems such as cloud spot markets, adaptive LLM inference, and order consolidation in logistics operate under capacity constraints. In these settings, decision-makers must allocate fixed resources among a stream of requests with heterogeneous reward rates and service durations. A critical tension arises when capacity is saturated: the system must decide whether to continue serving an active job to harvest its remaining value or to preempt it in favor of a new arrival. This trade-off is especially subtle when a new job offers a lower immediate reward rate but a longer service duration, potentially outweighing a high-rate job near completion.

To address this challenge, we introduce a general and parsimonious online allocation framework for capacity-constrained resources, where preemption is non-resumable and yields partial rewards. This abstraction captures the essential dynamics of modern resource management problems, including, but not limited to, those discussed earlier. We derive intuitive index-based policies and provide constant-factor competitive ratio guarantees for their performance, without requiring any forecasts of future demand. In particular, we show that two naive index-based policies have competitive ratios of $1/2$ and $1 - 1/e \approx 0.63$. Our main methodological contribution is to develop a flexible framework for defining "quasi-utilities" that couples the states of an online vs. offline algorithm, which reduces the design of online index-based policies to an offline geometrical problem. The framework allows us to optimize the index function, leading to an exponential-index-based policy that has an improved competitive ratio of 0.68. We also consider various extensions and prove that most of our analyses are tight. Finally, we conduct a concise numerical study to demonstrate the strong empirical performance of the proposed policies, under non-worst-case stochastic demand sequences.

Key words: online allocation, online scheduling, reusable resources, preemption, competitive ratio, cloud computing, personalization, index-based policies

History: This draft: August 3, 2026, forthcoming in Management Science. This paper had a former title "Real-Time Personalized Order Holding".

1. Introduction

High-throughput operational systems often function under capacity constraints, requiring decision-makers to manage finite resources against a continuous stream of online requests. This dynamic spans diverse modern application domains. For example, in cloud computing spot markets, providers sell excess computing capacity at steep discounts but retain the right to preempt these instances to serve higher-priority on-demand requests when capacity limits are hit (Pauley 2023). In adaptive LLM inference, KV-cache memory imposes a hard limit on the total number of tokens that can be stored during inference. Systems such as vLLM address this constraint through preemption: active sequences may be evicted to free KV-cache space and are recomputed from scratch when sufficient KV cache space becomes available again [1]. In e-commerce logistics, there is a limit on how many orders can be held and consolidated simultaneously before dispatch, due either to virtual constraints such as system-imposed pooling in a centralized buffer (Wang et al. 2024), or to physical constraints at downstream sortation systems such as put walls with limited number of cubbies (Boysen et al. 2019). Finally, in contingent workforce management, firms are constrained by the limited capacity of internal teams to handle incoming tasks, after which they must resort to outsourcing (Neubert and Adjadj (2009), Upwork[2]).

Despite their domain differences, these settings share a common operational tension: decision-makers must determine whether to continue allocating scarce capacity to ongoing work or interrupt it in order to accommodate new arrivals, even though doing so may irreversibly forgo future benefits from the ongoing work. In cloud computing, an early-terminated spot instance stops billing immediately. Because users typically must migrate the task or restart it later to avoid indefinite idle storage costs, such interruptions are effectively irreversible. The decision-maker must therefore determine whether to continue running active instances or to terminate them in favor of newly arrived requests that may offer higher priority or better utilization by keeping the system busy for longer. In adaptive LLM inference, where the reasoning process can be dynamically adjusted, thinking for longer generates a partially improved response whose value depends on the reasoning time. However, once the KV-cache state is discarded to free memory, the process cannot be resumed. The scheduler must therefore decide whether to allocate additional computation to further improve an existing response, or to terminate it early to admit new requests. In e-commerce logistics, retailers may temporarily hold customer orders (e.g., not seal the shipping box) for a certain period of time. Holding an order accrues value by increasing the likelihood of consolidating with other orders, but it also occupies scarce capacity (such as limited cubby space in a put wall)

[1] Link: `https://docs.vllm.ai/en/stable/configuration/optimization/#preemption`

[2] Link: `https://www.upwork.com/press/releases/upwork-inc-introduces-lifted-a-subsidiary-serving-enterprises-with-full-suite-of-contingent-workforce-solutions`

that could otherwise be allocated to new orders. If an order is dispatched early to free capacity, it forfeits any future consolidation opportunities. Finally, in contingent workforce management, firms must decide whether to continue allocating internal teams to ongoing projects to realize their remaining value, or to conclude them early (potentially transferring the unfinished work to external providers) in order to free capacity for higher-value incoming tasks.

All of these settings feature the same keep/switch trade-off, and our model is designed to capture it in a single, general form. The correspondence is most direct in the cloud-computing spot market, which we adopt as our primary motivating application and develop in detail in Section 1.2: the capacity limit, the per-unit-time billing reward, and non-resumable preemption each have literal counterparts. Downstream order consolidation at a put wall provides a second concrete setting in which the capacity constraint is especially tangible, since each open order occupies one of a finite number of cubbies (Section 2.3). The same trade-off also appears in adaptive LLM inference and contingent workforce management, which we discuss more briefly as additional illustrations of the model's broad applicability.

This *keep/switch trade-off* is fundamental yet nontrivial to accommodate, because it requires carefully balancing the rate at which value is currently generated against the remaining duration of service. A new arrival may offer a lower immediate reward rate than the active job, yet its longer service horizon could result in a higher total accumulated value. Prioritizing solely based on current rate ignores the opportunity cost of future capacity, while prioritizing solely based on duration risks crowding out high-value short-term tasks. In the following section, we formalize this keep/switch trade-off using a parsimonious online decision-making model.

1.1. Modeling the Keep/Switch Trade-off

This paper introduces a parsimonious online decision-making model for studying this keep/switch trade-off. In our core model, there is a fixed capacity C on the total number of requests that can be served. Requests $i = 1, \dots, N$ arrive over a continuous finite time horizon. Each request i has an arrival time t_i and a rate ("height") h_i at which reward is accumulated if that request is served. All requests can be served for a duration up to some maximum allowable amount or "lifespan," which we normalize to one, i.e., request i can be served until a time that can be as large as $t_i + 1$. If the system is at full capacity and a new request arrives, then a decision must be made on which request to preempt, possibly the request that has just arrived. The objective is to maximize the total reward accumulated from serving the requests.

While the core model formulates reward accumulation through a deterministic rate h_i, this abstraction captures a broad class of environments in which value accrues stochastically over time. For instance, in consolidation applications, cost savings may materialize only when subsequent

customer orders arrive, generating discrete reward increments at random times. As we discuss in Appendix A, such stochastic reward processes can be represented, in expectation, by equivalent deterministic reward-rate functions, allowing our framework to capture these dynamics without altering the structure of the online decision problem.

We elaborate on the trade-off in this problem (see Figure 1). Suppose $C = 1$ and the algorithm is currently serving request i which arrived at time t_i, and a new request j just arrived at the current time $t \in (t_i, t_i + 1)$. Keeping the current request i would accumulate reward at a rate of h_i; however, it can only be served until time $t_i + 1$, so the remaining reward to be extracted is $h_i \cdot (t_i + 1 - t)$. Meanwhile, preempting i and switching to the new request j would accumulate reward at a rate of h_j, with a maximum reward of $h_j \cdot 1$ that can be extracted.

If $h_j \geq h_i$, then it would be always better to serve request j, irrespective of what requests will arrive in the future. This is because request j would earn reward at a higher rate, and can also be served for a longer duration from the current time. On the other extreme, if $h_j \leq h_i(t_i + 1 - t)$, then it would be always better to serve request i, since it accumulates reward at a faster rate and also has more remaining reward to be extracted.

However, if $h_i(t_i + 1 - t) < h_j < h_i$, then we have a trade-off. On one hand, keeping request i maximizes the reward per unit time, but risks idling the service capacity if no other request arrives before time $t_i + 1$. On the other hand, switching to request j maximizes the reward to be extracted if no future requests arrive. In this trade-off situation, we say that request i has a greater "height" (since $h_i > h_j$) but request j has a greater "remaining area" (since $h_j \cdot 1 > h_i \cdot (t_i + 1 - t)$).

We also explore two model extensions. In the first extension (Section 6), we allow request to have heterogeneous lifespans, capturing heterogeneity in the service times. In the second extension (Section 7), we allow request to have time-varying rates at which reward is accumulated if that request is served.

1.2. Model Relevance for Cloud Computing Spot Markets

In this section, we illustrate the relevance of our model to the management of spot instances on cloud computing platforms such as AWS and Google Cloud, as well as to GPU rental marketplaces such as Vast.ai.

The recent AI wave has led to a surge in popularity for GPU rental services. Leading cloud platforms such as Amazon Web Services, Google Cloud, and Microsoft Azure offer spot instances, allowing users to utilize available excess computing capacity at a cost lower than the standard on-demand price. While spot instances are indeed cost-effective, they are also preemptible, subject to interruption whenever service providers need to reclaim capacity for higher-priority instances (e.g., on-demand and reservation instances). The demand for spot instances has surged since 2023,

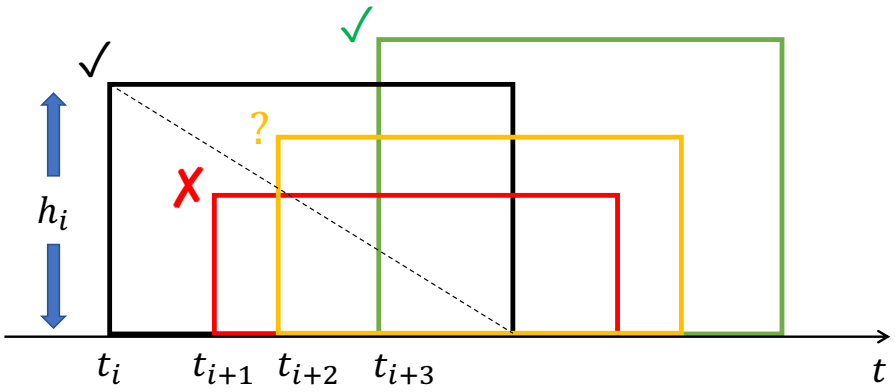


Figure 1 **Suppose $C = 1$ and the algorithm is currently serving request i with height h_i that arrived at time t_i. If another request arrives at time t_{i+1} with smaller remaining area (equivalently, the red request (t_{i+1}, h_{i+1}) is below the dashed black line), then the algorithm should continue to serve request i and reject request $i+1$. If a request arrives at time t_{i+2} and is above the dashed black line but has smaller height (i.e., the orange request), then the algorithm faces a non-trivial trade-off and the optimal action depends on future arrivals. Lastly, if a request arrives at time t_{i+3} with larger height (i.e., the green request), then the algorithm should start serving this new request $i+3$.**

resulting in a fourfold increase in preemptions (Pauley 2023). Faced with the escalating frequency of interruptions, service providers are compelled to determine which spot instances to preempt upon reaching capacity limits. When such interruptions occur, users have the option to either terminate the task or restart it once capacity becomes available. If terminated, service providers like AWS[3] and Google Cloud[4] still charge payments for the hours used, fitting the reward process in our model.

Vast.ai[5], as another example, is an emerging cloud computing marketplace that specializes in offering GPU rental services. It connects a wide range of GPU providers, from individual hobbyists to large data centers, with users seeking computational resources. The platform supports both on-demand and interruptible rental options. Interruptible instances operate on a bidding system: the instance with the highest bid runs. Similarly, when interrupted, users may prefer to terminate their task and migrate the remaining portion to another machine or service provider rather than wait indefinitely while incurring storage costs. As noted by Vast.ai: "once your instance is interrupted, it could be a long wait until it resumes." This practical tendency aligns with our model's assumption that tasks may not resume after interruptions. Users are also charged for the GPU hours used, again fitting the reward process in our model. It is also noteworthy that the lifespan here represents

[3] Link: `https://aws.amazon.com/ec2/spot/`

[4] Link: `https://docs.cloud.google.com/compute/docs/instances/spot`

[5] Link: `https://vast.ai/`

the total time required by the user to be fully served, beyond which they no longer need the service or are unwilling to pay for it. This time can be estimated from the specific task, or set directly by the user or provider. For instance, Google Cloud once imposed a 24-hour limit on spot instance usage, now customizable by users.

We now establish a precise mapping to our model. In these systems, users request computing resources that are served from a finite pool of GPUs, so that at any time the provider can run at most C interruptible instances in parallel, corresponding to the capacity constraint. Each active instance generates revenue at a rate h_i equal to its billing price per unit time, determined either by the user's bid in an interruptible marketplace or by the platform's spot pricing mechanism. Consequently, continuing to serve an instance yields revenue that accrues linearly over time.

Instances may be reclaimed by the provider when higher-priority demand arises, such as from on-demand or reserved instances. Upon interruption, the current execution of the task is terminated. Although users can in principle resume later, doing so typically requires provisioning a new instance and restoring from a checkpoint. In practice, because users incur storage costs while waiting for capacity and face uncertain resumption delays, it is often more economical to abandon the interrupted run and migrate the remaining workload to another machine or provider. As a result, the interrupted execution cannot be resumed without loss of progress, aligning the non-resumable preemption assumption in our model.

Furthermore, each instance is associated with an expected runtime or effective service horizon (determined either by the user's workload or by platform-imposed limits on spot usage) beyond which no additional value is generated. Preempting an instance therefore yields only the partial reward corresponding to the billed usage accrued up to the interruption time. Thus, the decision of whether to continue running an active instance or to reclaim it for a new arrival directly mirrors the keep/switch trade-off captured in our model.

1.3. Competitive Ratio Results

We now describe our theoretical results, starting with our core model where requests have constant reward rates and homogeneous lifespans. We consider online algorithms that do not require any forecast of the future. We analyze the competitive ratio (CR), which is the worst-case guarantee that an algorithm can provide on the total reward compared to that of an offline algorithm that knows the future in advance.

We start by analyzing π^{h}, an algorithm that always serves the requests with the largest heights, and π^{ra}, an algorithm that always serves the requests with the largest remaining areas. As discussed on the trade-off above, these two algorithms represent two extremes. We prove that π^{h} has a CR of exactly 1/2 (Proposition 1), i.e., it always accumulates total reward at least half of that of an

optimal offline algorithm, and moreover there exists an instance in which the algorithm collects exactly half. Meanwhile, we prove that π^{ra} has a CR of exactly $1-1/e$ (Proposition 2).

Next, we improve on the CR using algorithms that consider both height and remaining area when facing the trade-off. We provide a novel framework for deriving index-based algorithms and analyzing their CR's (Theorem 1). In particular, we show that an exponential-index-based algorithm $\pi^{\mathsf{exp}(\alpha^*)}$ has a CR of exactly $\frac{1-e^{-\alpha^*}}{\alpha^*}$, which exceeds 0.68 (Theorem 2) under an optimized parameter α^* designed by using the Lambert W function. This beats not only $1-1/e \approx 0.63$ but also an upper bound of 2/3 (Proposition 3) that limits any analysis based on a "Skyline" relaxation. We also provide upper bounds of 0.84 on the CR of any deterministic algorithm when $C=1$ (Proposition 4) and 0.92 on the CR of any randomized algorithm (Proposition 5).

Our first extension is to allow requests to have heterogeneous lifespans, letting L denote an upper bound on the ratio of the longest lifespan divided by the shortest lifespan. Here, we show that a competitive ratio which degrades as $O(1/\sqrt{L})$ is unavoidable (Proposition 6). We then derive a black-box reduction π^{red} that allows any policy π originating from the model with homogeneous lifespans (as described above) to be executed under heterogeneous lifespans, while losing a factor of at most $\sqrt{L}$ in the competitive ratio (Theorem 3). For example, if $\pi=\pi^{\mathsf{exp}(\alpha^*)}$, then the competitive ratio would be at least $0.68/\sqrt{L}$. We note that this holds even if $C=1$, and that the $O(1/\sqrt{L})$ upper bound also requires $C=1$. Motivated by this, we derive a second reduction algorithm π^{div} specifically for large-capacity regime, which leads to a slower competitive ratio degradation rate of $\Omega(1/\log(L))$ under the assumption that $C=\Omega(\log(L))$ (Theorem 4).

Our second extension is to allow requests to have time-varying reward rates, under a *homogeneity* assumption that the rewards of all requests scale at the same rate over time. To elaborate, we assume for a request i arriving at time t_i that its reward rate at time t is $h_i \cdot \mathfrak{r}(t-t_i)$, for all $t \in [t_i, t_i+1)$. We identify a condition on the scaling function $\mathfrak{r}$ that allows our exponential-index-based algorithm and analysis to automatically extend, with exponential functions $\{e^{-\beta t}\}_{\beta\in\mathbb{R}}$ being a class of functions that satisfy this condition. The competitive ratio depends on a parameter R, the integration of the scaling function $\mathfrak{r}(t)$ over $[0,1]$ (to be formally defined in (12)). When $R<1$, the competitive ratio *improves* beyond 0.68. In fact, $\mathcal{CR}$ goes all the way to 1 as R shrinks to 0. In the special case of exponential function $\mathfrak{r}(t)=e^{-\beta t}$, $\mathcal{CR}$ improves from 0.68 to 1 as the decay rate β increases from zero to infinity. The intuition for this result is that the online algorithm's trade-off becomes easier if the reward from serving requests decays so quickly, as the desiderata of "height" and "remaining area" coalesce upon one another.

Beyond these specific extensions, we establish that the underlying index-based framework is structurally robust to environments with substantial temporal variability in both system capacity and service quality. In particular, our competitive-ratio guarantees extend to settings with

time-varying capacity, where the total service capacity may fluctuate arbitrarily over time due to operational disruptions or shifting resource availability (see Remark 2). We further generalize our results to heterogeneous-resource environments (Appendix B), in which requests may be processed using various resource types with different service qualities. Notably, our policies retain their worst-case performance guarantees even when both capacity and service quality evolve dynamically over time (see Remark EC.1).

We provide an abridged summary of the competitive ratios discussed above in Table 1.

Table 1 An abridged presentation of our competitive ratio results in the core model and extensions. Here, $\mathcal{CR}(\pi)$ represents the competitive ratio of a policy π. The results are exact unless denoted by $\leq$ or $\geq$.

Setting	Competitive Ratio	Result	Reference
Core Model	$\mathcal{CR}(\pi^{\mathsf{h}})$	0.50	Proposition 1
	$\mathcal{CR}(\pi^{\mathsf{ra}})$	0.63	Proposition 2
	$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)})$	0.68	Theorem 2
	$\sup_\pi \mathcal{CR}(\pi)$ (Upper Bound, $C=1$)	≤ 0.84	Proposition 4
Heterogeneous Lifespans	$\mathcal{CR}\left(\pi^{\mathsf{red}}(\pi^{\mathsf{exp}(\alpha^*)})\right)$	$\geq 0.68/\sqrt{L}$	Theorem 3
	$\sup_\pi \mathcal{CR}(\pi)$ (Upper Bound, $C=1$)	$\leq 1/\sqrt{L}$	Proposition 6
	$\mathcal{CR}\left(\pi^{\mathsf{div}}(\pi^{\mathsf{exp}(\alpha^*)})\right)$ (when $C=\Omega(\log(L))$)	$\Omega(1/\log(L))$	Theorem 4
Time-Varying Rewards	$\mathcal{CR}\left(\pi^{\mathsf{exp}(\alpha^*(R))}\right)$	$\frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)}+1}$ ♣	Theorem 5

♣ We refer to Figure 3 on how $\alpha^*(R)$ and $\mathcal{CR}\left(\pi^{\mathsf{exp}(\alpha^*(R))}\right)$ change as a function of R, where R represents the integration of the scaling function $\mathfrak{r}$ over $[0,1]$ as defined in (12).

1.4. Sketch of Analysis Technique

At its heart, our analysis is based on the following argument. Suppose that the online algorithm always kept the (up to) C requests with the largest reward rates, and had the following privilege—if a request hits its deadline while being served by the algorithm, then its lifespan would be "extended" by one unit of time and would still be served using imaginary capacity. We show such an online algorithm upper-bounds the performance of any offline algorithm. Of course, this privilege is imaginary, and could overestimate the actual reward of the online algorithm by a factor of 2. Nonetheless, this argument immediately implies that the policy which keeps the C largest reward rates has a competitive ratio at least $1/2$.

Next, we observe that the above argument is preserved if rewards are replaced by "quasi-utilities"—an online algorithm that keeps the C largest quasi-utility rates and has the aforementioned privilege upper-bounds the quasi-utility that can be earned by any offline algorithm. If we make the definition of quasi-utility smaller than the actual reward, then the overestimation factor described above could be smaller, and its reciprocal which we denote by γ_a (one of the two key

factors in the competitive ratio guarantee) could be bigger. However, the optimal offline's quasi-utility would then be smaller than the optimal offline's actual utility, requiring us to multiply the guarantee by γ_o (the other key factor in the competitive ratio guarantee).

Importantly, this argument works for *any* definition of quasi-utility, and the factors γ_a and γ_o are easy to calculate from the quasi-utility function because they only compare online to online and offline to offline, respectively. In this way, our framework has achieved "disentanglement" of online and offline decisions. We highlight that our framework transforms the online decision-making problem to an offline geometrical problem, in which one shall design quasi-utility functions that have a large final competitive ratio $\gamma_a\gamma_o$, leading to an exponential quasi-utility function with an optimized decay rate. Our resulting algorithm will then be the policy that keeps the C requests with the highest indices according to these exponential functions.

Our flexible analysis framework allows us to define similar exponential quasi-utility functions in the extension where actual reward rates are time-varying, as we demonstrate in Section 7. We compare our analysis framework to existing literature in Subsection 2.4.

2. Literature Review

2.1. Online Scheduling

Our problem is closely related to the classical line of work on "competitive analysis" for online scheduling (see Kolen et al. 2007). Specifically, our model can be described as having sharp deadlines (jobs must be accepted or rejected immediately upon arrival) with no discounting in the model of Dasgupta et al. (2009), which allows for job cancellations with partial rewards. Note that Dasgupta et al. (2009) already contrasts with many other papers, such as Woeginger (1994), Canetti and Irani (1995), Epstein et al. (2016), which do not permit job cancellations with partial rewards. Our combination of features, namely the immediate start of jobs, fixed deadlines, continuous service periods, the possibility of early termination, and the option to receive partial rewards, has been largely overlooked in the classical literature, even though it is the norm in many modern applications as discussed in the Introduction. We also note that the trade-off we identify (see Figure 1) differs from the focus of these classical papers.

We next provide a concrete comparison to the results in Dasgupta et al. (2009). Assuming heterogeneous lifespans while maintaining a constant reward rate over time, Dasgupta et al. (2009) derive competitive ratio bounds with a logarithmic dependence on the term $\frac{\max_i h_i}{\min_i h_i}$, the ratio of the largest to smallest reward rates. Their algorithm essentially partitions the heights uniformly on a logarithmic scale, and then selects one of these height intervals (with a height ratio of 2) uniformly at random. Within the chosen interval, it disregards the specific heights and prioritizes the request with the latest deadline among the current requests in that interval. Intuitively, by being greedy

with respect to (w.r.t.) the lifespans, their algorithm's performance guarantee does not depend on L, the ratio of the longest to the shortest lifespan, but depends on $\frac{\max_i h_i}{\min_i h_i}$, the ratio of the largest to the smallest height. By contrast, our algorithm's guarantee depends on L but not on $\frac{\max_i h_i}{\min_i h_i}$. In particular, when $L=1$, we derive guarantees that exceed $1-1/e$, a naturally-occurring barrier in the analysis of similar problems. We note that in our applications, we generally expect $\frac{\max_i h_i}{\min_i h_i}$ to be large (due to heterogeneity across users), while L is likely to be small (due to similarity in delivery options), making our form of guarantee more appropriate for our problem. Our algorithm also has the benefit of being deterministic rather than randomized, and demonstrates empirical performance that significantly exceeds its worst-case guarantee.

Table 2 Implications of our core model (Subsection 3.1) in setting of Dasgupta et al. (2009) when resources do not expire (i.e., $\beta=1$ in Dasgupta et al. (2009)).

	Dasgupta et al. (2009)	Our Paper
Setting	Randomized Algorithms	Deterministic Algorithms
Lower Bound on CR	$\frac{1}{2\left\lceil \log \frac{\max_i h_i}{\min_i h_i} \right\rceil}$	$\geq 0.68/\sqrt{L}$ (for $C \geq 1$) $\Omega\left(1/\log L\right)$ (for $C=\Omega\left(\log L\right)$
Upper Bound on CR ($C=1$)	$O\left(\sqrt{\frac{\log\log L}{\log L}}\right)$	$\leq \min\left\{0.84,\ 1/\sqrt{L}\right\}$ ≤ 0.92 (for randomized alg.)

2.2. Online Resource Allocation with Reusability

Our paper also contributes to the recently growing literature on resource allocation with reusable resources in various contexts, such as ride-sharing (Dickerson et al. 2021), online matching (Goyal et al. 2020), pricing (Besbes et al. 2019, Lei and Jasin 2020), network revenue management (Baek and Ma 2022), assortment optimization (Rusmevichientong et al. 2020, Feng et al. 2022, Gong et al. 2022), crop gleaning (Ata et al. 2019), and volunteer crowdsourcing (Manshadi and Rodilitz 2020). Accept/reject decisions for reusable resources under adversarial arrivals are also studied in Huo and Cheung (2022), both with and without advance booking, and Ekbatani et al. (2024) considers the assignment problem with multiple servers. Abels et al. (2023) allow for selecting the duration of resource allocation at the time of the job's arrival. Golrezaei and Yao (2025) consider a setting where some requests can wait for a single unit of time if their service is guaranteed. In all of the extant literature, once a reusable resource is assigned to a job, it cannot be reassigned to a new job until the current job is completed. By contrast, in our model/application, the server can preempt current requests at any time to make room for more rewarding requests while still collecting partial rewards for the duration of service provided.

2.3. E-commerce Fulfillment

Beyond cloud computing, e-commerce fulfillment serves as another key motivating application, as discussed in the Introduction. The rapid growth of e-commerce has driven the emergence of large-scale, highly automated fulfillment systems, where real-time optimization algorithms play a central role in enabling order consolidation and reducing fulfillment costs (Qin et al. 2022). Wu (2018) document the practice of personalized order holding at JD.com. Wei et al. (2021) investigate the optimal consolidation policy in the setting of two warehouses and capture the trade-off between economies of scale from consolidating orders and the expedited shipping costs induced by shorter deadlines. Chen et al. (2024) model customer-level order holding as a Markov decision process and show that the optimal release policy follows a threshold structure. Feng et al. (2024) study how to consolidate truckloads in the middle mile and show using data from Alibaba Cainiao that adding a small amount of flexibility to the routing network can significantly reduce transportation costs.

Wang et al. (2024) is the most closely related work to our setting in this domain. They formulate the order consolidation problem as a dynamic program in which newly arrived orders are held in a temporary pool subject to a finite holding capacity, and release timing is optimized to balance consolidation benefits against deadline-induced shipping costs. However, solving their model exactly is computationally intractable due to the curse of dimensionality, motivating the use of LP-based heuristic approximations. In contrast, we adopt a parsimonious abstraction that captures the fundamental trade-off between holding newly arrived orders with low consolidation probability and continuing to hold orders that have already accrued consolidation value while occupying scarce capacity. This allows us to develop intuitive online policies with worst-case performance guarantees, whereas existing approaches rely on heuristic approximations without such guarantees.

Our work also contributes to the literature on downstream fulfillment control, where retailers decide whether to temporarily hold orders to enable consolidation before dispatch. During this holding period, each order generates value at a rate reflecting the expected marginal shipping-cost savings from potential future consolidation. We emphasize that our model captures one specific abstraction of consolidation value—the expected marginal shipping-and-handling savings that accrue while an order is held—rather than the full operational fulfillment problem, in which realized savings also depend on future stochastic order arrivals, parcel composition, routing, and downstream constraints. Appendix A makes precise the sense in which such a stochastic reward can be represented, in expectation, by an equivalent deterministic reward rate.

This physical holding process is implemented through a sortation system known as a put wall, which consists of a strictly finite number of cubbies that serve as temporary holding positions for open orders. Because each open order must occupy a dedicated cubby, the total number of orders that can be accumulated simultaneously is strictly bounded by this physical capacity (Boysen et al.

2019). In this sense, put-wall consolidation materially realizes the capacity constraint, in contrast to the upstream virtual-pooling setting (Wang et al. 2024), where such constraints arise in a more abstract or "soft" operational form.

More broadly, order holding enables several operational benefits, including creating a backlog that facilitates improved sequencing decisions, increasing market thickness, allowing order cancellations and consolidations, and providing additional information to online decision-makers (see Xie et al. (2024) for a classification). It also helps mitigate challenges associated with multi-item orders (where a single order containing multiple items may otherwise be split into multiple packages) in e-commerce (see Zhao et al. (2025)). In contrast to Xie et al. (2024), who emphasize informational benefits, our work focuses on consolidation benefits and models them explicitly through a holding reward that reflects the expected savings from future order arrivals.

Finally, the keep/switch trade-off studied in this paper, where an active instance may be intentionally preempted to accommodate newly arrived instances that could yield higher overall system efficiency, is similar in spirit to the use of demand rejection as a control mechanism in inventory networks. In particular, Yang et al. (2025) show that fulfilling all incoming demand can induce significant spillovers across the e-retailer's network, and that selectively rejecting sales can improve overall performance. A related mechanism arises in omnichannel fulfillment through booking-limit controls, where retailers may defer the fulfillment of online orders from local customers to more distant warehouses in order to preserve local capacity for future in-store demand, thereby improving overall system performance (Gao et al. 2022).

2.4. Comparison of Techniques

We compare our analytical framework, outlined in Subsection 1.4, to existing techniques in the literature. There are papers (e.g., Chrobak et al. 2003, 2007, Gong et al. 2022, Simchi-Levi et al. 2025) that establish competitive ratios by directly comparing the optimal offline algorithm's reward against that of the algorithm. This charging argument creates a many-to-one mapping of partitioned rewards, across time, demand requests, or resources. For instance, by ensuring that each reward unit of OPT corresponds to at most $\frac{1}{\mathcal{CR}}$ copies of the online algorithm's rewards for some $\mathcal{CR} \leq 1$, a competitive ratio of $\mathcal{CR}$ is established.

Another common technique for establishing competitive ratios in online algorithms is primal-dual approach. Here, the goal is to compare the algorithm's performance to the dual of a natural LP upper bound for the optimal offline value. The algorithm's rewards are partitioned among the dual variables so that the algorithm achieves the same total value by construction, after which one argues that these dual variables approximately satisfy feasibility (e.g., Devanur et al. 2013). An alternative approach is to guarantee dual feasibility while arguing that the algorithm's gain

is at least $\mathcal{CR}$ times the dual gain at every time step (e.g., Buchbinder et al. 2013). For both alternatives of the primal-dual framework, we note that exponential index functions naturally arise as a by-product (e.g., Mehta et al. 2007, Golrezaei et al. 2014).

Nonetheless, our proof technique is tailored to our specific problem and does not directly align with any known versions of the charging or primal-dual arguments. To recap, we introduce a quasi-utility metric and an "extended version" of any policy that is specific to our problem, after which there are three layers to our argument. The first layer shows that the proposed policy's actual utility approximates the quasi-utility of its extended policy. The second layer establishes that the extended policy's quasi-utility exceeds that of the optimal offline. The third layer demonstrates that the optimal offline's quasi-utility approximates its actual utility. In addition, we provide a geometrical interpretation, reducing the complex online decision-making problem into an "offline" geometry problem, involving a parallel structure on a plane (see Subsection 4.2 for further details).

3. Model and Simple Index-based Policies

In this section, we formally introduce our model and notation, and start by analyzing some simple index-based policies. All proofs in the paper will be deferred to the appendix.

Let $[N] \triangleq \{1, 2, \ldots, N\}$. An instance of our problem is given by $\mathcal{I} = \left(N, \{(t_i, h_i, \ell_i)\}_{i\in[N]}\right)$, where N denotes the total number of requests arriving to the system. For each request $i \in [N]$, t_i is its arrival time, h_i is its height (reward rate), and ℓ_i is its lifespan. We allow multiple requests to arrive simultaneously. The lifespan ℓ_i of request i is the maximum duration for which the system may serve the request after its arrival at time t_i. We refer to the time $t_i + \ell_i$ as the deadline (or end of lifespan) of request i, and request i is said to be "active" during interval $[t_i, t_i + \ell_i)$. We can also define $T \triangleq \max_{i\in[N]}\{t_i + \ell_i\}$ as the time horizon of the instance $\mathcal{I}$, since all requests must have been served by that time. All requests will be realized in an online fashion, and both N and T are unknown to the system (i.e., decision-maker).

3.1. Core Model

We next present the *core model*, which serves as the basis for our analysis. Its assumptions are primarily introduced to simplify exposition and analysis; as we show later, our results extend to more general settings, including those in Sections 6 and 7. We define the *core model* as the special case in which all lifespans are identical, and our analysis focuses on this setting until we introduce extensions in Sections 6 and 7. We next state the three key assumptions in our core model.

First, we assume that all lifespans are identical. This assumption is mainly driven by business constraints; for example, Google Cloud imposes a 24-hour limit on spot instance usage, and one-day shipping orders are often subject to fixed holding windows (e.g., up to 30 minutes). By re-scaling

time, we normalize them all to one, i.e., $\ell_i = 1 \ \forall i \in [N]$. Later on, we relax this assumption and study the setting with heterogeneous lifespans ℓ_i in Section 6.

Second, we assume that the system has a capacity C in the sense that we cannot serve more than C number of requests at any point in time. This assumption captures practical capacity constraints, such as the number of machines available in cloud computing or the number of cubbies in a put-wall consolidation system, as discussed in the Introduction and Subsection 1.2. In Remark 2, we further state that our analysis extends to settings in which the capacity C varies arbitrarily over time.

Third, the reward rate h_i is constant over time. This assumption is well motivated in practice; for example, in cloud computing, a customer's willingness to pay is typically stable over the duration of a session. We later relax this assumption and study the setting in which the reward rate of serving any request can vary over time in Section 7.

REMARK 1. The reward rate is assumed to be deterministic in this paper, either time-invariant (core model) or time-varying (Section 7). This assumption applies to scenarios where the reward process can be modeled as the accumulation of a known, user-specific reward rate over time. For instance, in cloud computing, this corresponds to the price per unit of computation time. In contrast, some applications, such as order consolidation, may involve stochastic reward processes (e.g., multiorder cost savings modeled as jump processes). As explained in Appendix A, such stochastic reward processes can be reformulated as equivalent deterministic reward processes by defining appropriate reward rates, as long as the stochastic reward process for serving each request has known (time-varying) means that are fixed upon arrival and do not depend on any realizations.

We formally define a policy π (that could be online or offline) as follows. Let us first define $\mathcal{H}(t)$ as the set of indices for the requests that are being served in the system at time t, which we refer to as the service set at time t. In addition, we define $\mathcal{A}(t)$ as the set of requests that are arriving at time t, i.e., $\mathcal{A}(t) \triangleq \{i | t_i = t\}$. A policy π can serve/preempt any of those already arrived requests, i.e., $\mathcal{H}(t) \subseteq \{\cup_{\tau \le t} \mathcal{A}(\tau)\}$ for all t. π is called feasible if it satisfies the following constraints for all t:

$$\mathcal{H}(t) \subseteq \mathcal{H}(t^-) \cup \mathcal{A}(t), \tag{1}$$

$$|\mathcal{H}(t)| \le C, \tag{2}$$

$$\mathcal{H}(t) \bigcap \left\{ i \in [N] \middle| t_i + 1 \le t \right\} = \varnothing, \tag{3}$$

where t^- refers to the infinitesimal instant in time right before t. Here, (1) states that at any time t, π can only serve a subset of what it was serving right before time t as well as the newly-arrived requests. This constraint ensures that preemption is irreversible, a property that arises naturally in our motivating applications, including the termination of cloud spot instances, the clearing of the KV cache in adaptive LLM inference, and the dispatch of customer orders in order

consolidation (as discussed in the Introduction and Subsection 1.2). (2) is the capacity constraint, and (3) ensures that no request can be served in the system after reaching its deadline. We denote the set of all feasible policies by Π_f. We call a policy online if its decisions on which requests to preempt do not rely on future information. More formally, its decision at time t only depends on $\left\{\{\mathcal{A}(t')\}_{t'\in[0,t]},\ \{\mathcal{H}(t')\}_{t'\in[0,t)}\right\}$. We also denote by Π_f^{on} $(\subseteq \Pi_f)$ the set of all online feasible policies.

In the following we define additional notations that will help us to study and analyze our model. Given any policy and arrival instance of requests, we define d_i as the departure time of the request i; i.e., $d_i = \sup_t \left\{t \in [t_i, t_i+1) \middle| i \in \mathcal{H}(t)\right\}$, with $d_i = t_i$ if i was never served. With all this in mind, we can formulate the utility of any policy π in a system with capacity C under any instance $\mathcal{I}$, as:

$$U_\pi(\mathcal{I}; C) \triangleq \int_0^T \left(\sum_{i\in\mathcal{H}(t)} h_i\right)\ dt = \sum_{i\in[N]} h_i(d_i - t_i). \tag{4}$$

Note that both $\mathcal{H}(t)$ and d_i depend on π, and we suppress this dependence for the simplicity of notations.

Let $\mathsf{OPT}(\mathcal{I}; C) \triangleq \max_{\pi\in\Pi_f} U_\pi(\mathcal{I}; C)$ denote the maximum total reward that can be extracted from instance $\mathcal{I}$ by a policy π that can be either offline (i.e., knowing the future) or online, but must still be feasible (i.e., satisfying conditions (1)–(3) above). For ease of notation, we drop the dependence on C and write $U_\pi(\mathcal{I})$ and $\mathsf{OPT}(\mathcal{I})$ as the shortened versions of $U_\pi(\mathcal{I}; C)$ and $\mathsf{OPT}(\mathcal{I}; C)$, respectively, and only make this dependence explicit as needed (e.g., in Section 6.2). We also refer to the optimal offline policy by OPT for convenience. Additionally, let $\arg\max^C$ be the operator that returns the indices (request's number) for the C largest elements in a set (that tie-breaks in favor of larger indices, and returns the entire set if it has fewer than C elements). We also define $S_t^C \triangleq \arg\max_i^C\{h_i | \forall i : t_i \le t < t_i + 1\}$, which consists of the indices of the (at most) C requests with largest reward rates among all requests that are active at time t.

We further define the following *Skyline* relaxation from OPT as:

$$\mathsf{Skyline}(\mathcal{I}) \triangleq \sum_i h_i \left(\int_{t_i}^{t_i+1} \mathbb{1}[i \in S_t^C]\ dt\right) = \int_0^T \left(\sum_{i\in S_t^C} h_i\right)\ dt.$$

Here, we relax constraint (1), i.e., $\mathsf{Skyline}$ allows re-serving a request that was previously preempted as long as its deadline has not been reached, whereas OPT cannot re-serve a request. With this in mind, it is not hard to see that $\mathsf{Skyline}$ always serves the (up to) C active requests with largest reward rates. Thus, for any policy π and instance $\mathcal{I}$, we have:

$$U_\pi(\mathcal{I}) \le \mathsf{OPT}(\mathcal{I}) \le \mathsf{Skyline}(\mathcal{I}).$$

We use *competitive ratio* as our performance metric when comparing online policies against the optimal offline OPT (or, similarly defined, against the Skyline relaxation $\mathsf{Skyline}$). More formally, we define competitive ratio of an algorithm π w.r.t. OPT as follows:

$$\mathcal{CR}(\pi) \triangleq \inf_{\mathcal{I}} \frac{U_\pi(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})}.$$

We note that $\mathcal{CR}(\pi)$ both provides a guarantee on the fraction of $\mathsf{OPT}(\mathcal{I})$ that is obtained by algorithm π on any instance $\mathcal{I}$, and also indicates that there exists a worst-case instance on which this fraction is exactly $\mathcal{CR}(\pi)$. We also conduct a concise numerical study in Section 8 to evaluate the average performance of proposed algorithms under non-worst-case demand sequences.

3.2. Index-Based Policies

In this section, we introduce the family of *index-based policies*. This class of policies forms the foundation of our robust performance guarantees. Each policy in this family is associated with an index (or score) function that dynamically evaluates the priority of a request at any given moment; the policy then simply compares these indices to make real-time retention and preemption decisions.

Formally, let $f_z : \mathbb{R} \to \mathbb{R}$ be an index function associated with a request $z \triangleq (t, h)$, which is characterized by its arrival time t and intrinsic height h. The value $f_z(t')$ represents the real-time index score of request z at any time $t' \in \mathbb{R}$.

Definition 1 (**Index-Based Policy**). An index-based policy, defined by a family of index functions $\{f_z(\cdot)\}_{z\in\mathbb{R}^2_+}$, operates such that at any time t, the set of served requests $\mathcal{H}(t)$ is given by:

$$\mathcal{H}(t) = \arg\max^C \{f_{z_i}(t) \mid i \in \mathcal{H}(t^-) \cup \mathcal{A}(t) \setminus \mathcal{A}(t-1)\}.$$

To clarify the operational mechanics of this mathematical definition, Algorithm 1 outlines the exact step-by-step procedure the system executes whenever its state changes.

In words, the policy continuously purges expired requests and maintains a priority queue of the remaining candidates based on their instantaneous index scores. Whenever capacity is strictly saturated ($> C$), it greedily protects the C requests with the highest scores and discards the rest. We highlight that because this mechanism relies solely on current index values and the states of active requests, any index-based policy inherently constitutes a feasible, non-anticipatory online policy.

To motivate our design of index functions, recall the following discussion from the Introduction. Given a request i that is currently being served which arrived at time t_i with height h_i, and a newly-arrived request at the current time $t \in (t_i, t_i + 1)$ with height h, it is always better to serve the new request if $h \geq h_i$, and always better to continue to serve request i if $h_i(1 + t_i - t) \geq h$. Therefore, we focus on the regime where $h_i(1 + t_i - t) \leq h \leq h_i$, in which case we have a trade-off. One sensible approach is to first analyze the two extreme index functions for making this trade-off:

Algorithm 1 Base Index-Based Policy

At any time t when a system event occurs (i.e., a new request arrives or an active request reaches its maximum lifespan):

1. **Expiration:** Remove any active request $i \in \mathcal{H}(t^-)$ that has reached its deadline (i.e., $i \in \mathcal{A}(t-1)$).

2. **Candidate Pool Generation:** Let $\mathcal{P}(t)$ be the pool of candidate requests, consisting of the remaining active requests currently served in the system plus any new requests arriving at time t.

3. **Sorting and Allocation:**

- If $|\mathcal{P}(t)| \le C$, admit/retain all requests in $\mathcal{P}(t)$.
- If $|\mathcal{P}(t)| > C$, retain the C requests in $\mathcal{P}(t)$ possessing the highest current index values $f_{z_i}(t)$. Irreversibly preempt (or reject) the remaining $|\mathcal{P}(t)| - C$ requests.

- the extreme where the index is the height of the request, i.e. $f_z(t') = h$, implying that the algorithm would always preempt the newly-arriving request (with lower height) when there is a trade-off;
- the extreme where the index is the remaining area of the request, i.e. $f_z(t') = h(1 + t - t')$, implying that the algorithm would always preempt the currently-served request (with lower remaining area) when there is a trade-off.

These warm-up algorithms are analyzed in Subsections 3.3 and 3.4 respectively. Afterward we move toward a more sophisticated family of algorithms (called $\pi^{\mathcal{B}}$) in Section 4, which aim to balance between the two extremes.

3.3. Policy that Serves the Requests with Largest Heights

Our first algorithm always serves the (up to) C requests with the highest reward rates.

Definition 2 (π^{h}**: Serve Highest Heights)**. π^{h} is the index-based policy with index function $f_z(t') = h$ for all $t' \in \mathbb{R}$.

π^{h} is both intuitive and efficient, making it appealing and straightforward to implement in practice. For example, as previously mentioned, it is utilized in Vast.ai's interruptible instance rental model. It achieves at least $\frac{1}{2}$ competitive ratio against the optimal offline OPT, as well as the stronger benchmark $\mathsf{Skyline}$.

Lemma 1. *For any instance $\mathcal{I}$, we have*

$$2\, U_{\pi^{\mathsf{h}}}(\mathcal{I}) \ge \mathsf{Skyline}(\mathcal{I}) \ge \mathsf{OPT}(\mathcal{I}).$$

The following proposition states that the competitive ratio of policy π^{h} is exactly $\frac{1}{2}$, i.e. π^{h} can earn no better than 1/2 of the optimum, in a worst-case instance.

PROPOSITION 1. *The competitive ratio of policy* π^{h} *is*

$$\mathcal{CR}(\pi^{\mathsf{h}}) = \frac{1}{2}.$$

3.4. Policy that Serves the Requests with Largest Remaining Areas

Our second algorithm always serves the (up to) C requests with the largest remaining areas.

DEFINITION 3 (π^{ra}**: Serve Largest Remaining Areas)**. π^{ra} is the index-based policy with index function $f_z(t') = h(t+1-t')$ for all $t' \in [t, t+1]$ and is equal to zero outside of this interval.

π^{ra} is also intuitive and simple to implement, and has a higher competitive ratio than π^{h}.

PROPOSITION 2. *The competitive ratio of policy* π^{ra} *is*

$$\mathcal{CR}(\pi^{\mathsf{ra}}) = 1 - \frac{1}{e} \geq 0.63.$$

4. Framework for Designing and Analyzing a Family of Index-based Policies

In this section, we introduce and analyze a new family of index functions, which aim to make more balanced decisions than the two extreme policies $\pi^{\mathsf{h}}, \pi^{\mathsf{ra}}$.

4.1. General Framework based on Continuous Partitioning and Base Curves

We first introduce the notion of "continuous partitioning" of $\mathbb{R}^2$. Define a collection of curves $\{\mathcal{B}_r(.)\}_{r\in\mathbb{R}}$ (called "Base Curves"), where $\mathcal{B}_r(.) : \mathbb{R} \to \mathbb{R}$. We say they are a continuous partitioning of $\mathbb{R}^2$ if:

1. for any $r \in \mathbb{R}$, $\mathcal{B}_r(.)$ is a continuous function with $\mathcal{B}_r(0) = r$;
2. each point $z = (t, h) \in \mathbb{R}^2$ falls on exactly one of these functions, whose subscript is denoted by r_z; in other words, r_z is the unique value for which $h = \mathcal{B}_{r_z}(t)$.

One such example is $\mathcal{B}_r(t) \triangleq re^{-t}$, and it is easy to verify that the aforementioned two properties are satisfied. Note that $\mathcal{B}_r(.)$ does not have to be decreasing, and it can even be non-monotonic. For convenience, we sometimes refer to the continuous partitioning $\{\mathcal{B}_r(.)\}_{r\in\mathbb{R}}$ as just $\mathcal{B}$.

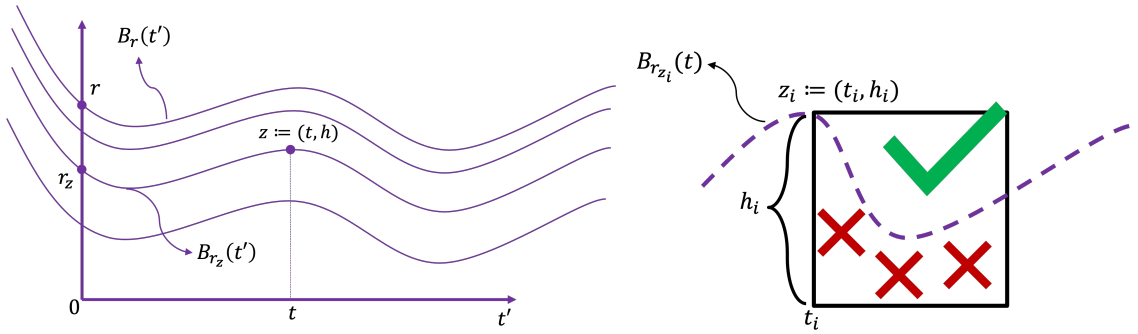
The properties of continuous partitioning guarantee that the base curves must have a parallel structure, in the sense that if a curve is higher than another one at some time t, then it must be higher at all times. Combining this observation with the definition that $\mathcal{B}_r(0) = r$, we have

$$r \geq r' \iff \mathcal{B}_r(t) \geq \mathcal{B}_{r'}(t)\ \forall t, \tag{5}$$

a fact to be used in our analysis.

We now introduce a family of index-based algorithms associated with $\mathcal{B}$.

DEFINITION 4 ($\pi^{\mathcal{B}}$**: Index-Based Policy with Base** $\mathcal{B}$**)**. $\pi^{\mathcal{B}}$ is the index-based policy with index function $f_z(t') = \mathcal{B}_{r_z}(t')$ for all $t' \in \mathbb{R}$.



(a) An example of continuous partitioning (b) $\pi^{\mathcal{B}}$: corresponding index policy with base $\mathcal{B}$

Figure 2 An illustration of continuous partitioning and index-based policy.

We illustrate these concepts in Figure 2.

We next define two key parameters associated with $\mathcal{B}$:

$$\gamma_o \triangleq \inf_{z=(t,h)\in\mathbb{R}_+^2,\ t'\in(t,t+1)} \left\{ \frac{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''}{h(t'-t)} \right\}, \tag{6}$$

$$\gamma_a \triangleq \inf_{z=(t,h)\in\mathbb{R}_+^2} \left\{ \inf_{t'\in(t,t+1)} \left\{ \frac{h(t'-t)}{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''} \right\}, \frac{h}{\int_t^{t+2} \mathcal{B}_{r_z}(t'')\ dt''} \right\}. \tag{7}$$

Note that if all the base curves $\mathcal{B}_r(.)$ are non-increasing, then the inner inf operator in (7) would be equal to one (minimized when t' is arbitrarily close to t, recalling that $\mathcal{B}_{r_z}(t) = h$), greatly simplifying the expression of γ_a. In fact, the exponential base curves to be discussed in Section 4.3 satisfy this property. Regarding the calculation of these two parameters, we are able to explicitly calculate them in the special case of exponential base curves (as stated in Lemma 5). Additionally, for other potential base curves with "good" properties (e.g., Lipschitz continuous in the domain of the problem), we can efficiently compute them numerically with a grid search, because the dimension of the minimization problems in (6) and (7) are three.

We now state the key result of the paper, which provides a way to explicitly calculate a lower bound on the competitive ratio of $\pi^{\mathcal{B}}$ for any $\mathcal{B}$. Here, the lower bound involves γ_a and γ_o.

THEOREM 1. *For any base $\mathcal{B}$, the competitive ratio of policy $\pi^{\mathcal{B}}$ is at least $\gamma_a\gamma_o$:*

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a\gamma_o \mathsf{OPT}(\mathcal{I}) \qquad \forall \mathcal{I}.$$

One key component in our argument is to decouple the entanglement between $\pi^{\mathcal{B}}$ and the optimal offline OPT, by breaking their comparison into three separate parts that we study in Lemmas 2, 3, and 4 below. More specifically, Lemma 2 compares the utility under policy $\pi^{\mathcal{B}}$ with some variant of the utility (called "quasi-utility") under an adjusted version of policy $\pi^{\mathcal{B}}$ (called "extended" policy $\tilde{\pi}$) in the "Policy space". Lemma 4 does a similar comparison in the "Optimal-offline space"

by comparing the quasi-utility under the optimal offline with its actual utility. The construction of γ_a and γ_o come naively from the comparisons. Last but not least, Lemma 3 bridges the two (disentangled) worlds by relating the quasi-utility under the optimal offline OPT to that under the extended policy $\tilde{\pi}$.

We also hope to highlight that the ratios of both Lemmas 2 and 4 (i.e., γ_a, γ_o) only rely on the geometrical structure of the base $\mathcal{B}$. Accordingly, this framework can transform the problem of finding improved competitive ratios for an online decision-making problem to an offline geometry problem in which we aim to find the best (w.r.t. the problem's specifics) continuous partitioning of $\mathbb{R}^2$ that returns a higher multiplicative factor of $\gamma_a\gamma_o$.

Remark 2 (Robustness to Time-Varying Capacity). While our primary exposition assumes a constant homogeneous capacity C for clarity, all competitive ratio guarantees derived for our index-based policies π^B naturally extend to environments with dynamic capacity. Specifically, our results hold even if the available system capacity is governed by an adversarially chosen function $C(t) : \mathbb{R} \to \mathbb{Z}^+$, which restricts both our online algorithm and the offline optimum (OPT) equally at any given time t. In the event of a capacity reduction, the adapted online policy simply preempts the currently-served request (or requests) with the lowest index value until the new capacity constraint is met. This robustness is a direct mathematical consequence of our unified analytical framework, specifically the parallel partitioning technique and the instantaneous, pointwise evaluation inherent in the index-based structure of our policies.

4.2. Proof of Theorem 1

In this subsection, we prove Theorem 1. As discussed earlier, we break the proof into three parts.

We first introduce a key concept - an imaginary policy $\tilde{\pi}$ (called "extended" policy of π) that is built upon policy π. It is possibly infeasible (both capacity-wise and deadline-wise) and its usage is solely for the purpose of analyzing the competitive ratio between the original policy π and optimal offline OPT. Given a feasible policy π, we define its extended policy $\tilde{\pi} \triangleq \tilde{\pi}(\pi)$ as follows (we usually just write $\tilde{\pi}$ for brevity).

Definition 5 ($\tilde{\pi}$**: Extended Policy of** π**)**. Consider a policy π, which removes request i at time $d_i, \forall i \in [N]$. Then, we define the departure times under $\tilde{\pi}$ (the extended policy of π) as follows. For any request i, if $d_i < t_i + 1$, $\tilde{\pi}$ will also remove this request at time d_i; otherwise (if $d_i = t_i + 1$), $\tilde{\pi}$ will serve the request for an additional unit of time. More formally, let $\tilde{d}_i$ denote the departure time of request i under $\tilde{\pi}$. Then,

$$\tilde{d}_i = \begin{cases} d_i & \text{if } d_i < t_i + 1; \\ d_i + 1 & \text{if } d_i = t_i + 1. \end{cases}$$

We also introduce a notion of "quasi-utility" for any policy π defined by departure times $\{d_i\}_{i\in[N]}$, denoted by $V_{\pi,\mathcal{B}}(\mathcal{I})$, similar to the definition of utility $U_\pi(\mathcal{I})$ in (4) with the difference that the reward is accumulated at the rate of the base $\mathcal{B}$ corresponding to each request instead of its height. We emphasize that this definition is valid even for policies such as $\tilde{\pi}$ which may not be feasible. In particular, if $\tilde{d}_i = t_i + 2$ for some request i, then $\tilde{\pi}$ would collect reward from i at the rate given by the base curve for a duration of length 2.

DEFINITION 6 ($V_{\pi,\mathcal{B}}$**: Quasi-Utility of** π **w.r.t. base** $\mathcal{B}$). The quasi-utility of a policy π w.r.t. base $\mathcal{B}$ under instance $\mathcal{I}$ is defined as:

$$V_{\pi,\mathcal{B}}(\mathcal{I}) \triangleq \sum_{i\in[N]} \left(\int_{t_i}^{d_i} \mathcal{B}_{r_{z_i}}(t)\ dt \right).$$

Finally, we think of OPT as a policy that preempts each request i at time d_i^*. We are now ready to begin our analysis, which compares the true and quasi-utilities of policies $\pi^{\mathcal{B}}$, $\widetilde{\pi^{\mathcal{B}}}$ (the extended policy of $\pi^{\mathcal{B}}$), and OPT.

The following lemma first compares the actual utility of index-based policy $\pi^{\mathcal{B}}$ with the quasi-utility of its extended policy $\widetilde{\pi^{\mathcal{B}}}$.

LEMMA 2. *For any instance* $\mathcal{I}$ *and base* $\mathcal{B}$*, we have*

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}).$$

Our next lemma compares the quasi-utility of the extended policy $\widetilde{\pi^{\mathcal{B}}}$ with that of OPT.

LEMMA 3. *For any instance* $\mathcal{I}$ *and base* $\mathcal{B}$*, we have*

$$V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}) \geq V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}). \tag{8}$$

Intuitively, Lemma 3 says that a policy which keeps the requests with the highest base curves collects at least as much quasi-utility as any other policy, including OPT, if it has the further privilege of "extending" the departure time of any request that is being served and reaches its original deadline. Here, we provide a proof sketch for this key idea. We first change the order of summation with integration and show that the summation of the base curve values for the requests that $\widetilde{\pi^{\mathcal{B}}}$ is serving is at least that of the requests that OPT is serving at any time t:

$$\sum_{i\in\mathcal{H}_{\widetilde{\pi^{\mathcal{B}}}}(t)} \mathcal{B}_{r_{z_i}}(t) \geq \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t).$$

We then note that, at any time t, any request that is being served by $\pi^{\mathcal{B}}$ is also being served by $\widetilde{\pi^{\mathcal{B}}}$. We next prove a claim that for any request $z = (t, h)$, if its base value $\mathcal{B}_{r_z}(t')$ at some time $t' \in [t, t+1)$ is lower than all the base values of the requests that $\pi^{\mathcal{B}}$ is serving at t', then $\mathcal{B}_{r_z}(t'')$

must be lower than each of the C largest base values that $\widetilde{\pi^{\mathcal{B}}}$ is serving at that time t'', for all $t'' \in [t', t'+1)$. Finally, by partitioning the set of requests that OPT is serving at time t into two sets - its intersection with the ones that $\pi^{\mathcal{B}}$ is serving as well as its difference set - and using the result of the claim, we show that each of the C largest base values of the requests that $\widetilde{\pi^{\mathcal{B}}}$ serves is (weakly) larger than each of the base values in the difference set.

Our final lemma compares the quasi-utility of OPT with the actual utility of OPT.

LEMMA 4. *For any instance $\mathcal{I}$ and base $\mathcal{B}$, we have*

$$V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) \geq \gamma_o \mathsf{OPT}(\mathcal{I}).$$

4.3. Exponential Base Curves

In this subsection, we analyze a specific family of base curves that satisfy the two properties of continuous partitioning, and explicitly calculate the corresponding γ_a and γ_o.

Define the following family of exponential base curves w.r.t. the parameter $\alpha \in \mathbb{R}$:

$$\forall r, t \in \mathbb{R}, \quad \mathcal{B}_r^{\exp(\alpha)}(t) = re^{-\alpha t}. \tag{9}$$

It is easy to see that $r_z = he^{\alpha t}$. We call $\mathcal{B}^{\exp(\alpha)}$ the *exponential base*, and denote by $\pi^{\exp(\alpha)}$ its associated base policy (i.e., the policy $\pi^{\mathcal{B}}$ in Definition 4 with $\mathcal{B} = \mathcal{B}^{\exp(\alpha)}$).

We now focus on analyzing the exponential base $\mathcal{B}^{\exp(\alpha)}$. The following lemma explicitly computes the γ_a and γ_o associated with the exponential base $\mathcal{B}^{\exp(\alpha)}$.

LEMMA 5. *For any $\alpha \geq 0$, the γ_a and γ_o associated with the exponential base $\mathcal{B}^{\exp(\alpha)}$ are*

$$\gamma_a = \min\left\{1, \ \frac{\alpha}{1-e^{-2\alpha}}\right\}, \quad \gamma_o = \frac{1-e^{-\alpha}}{\alpha}.$$

Combining Lemma 5 with Theorem 1, we derive a lower bound on the competitive ratio of $\pi^{\exp(\alpha)}$, which equals $\gamma_a\gamma_o = \min\left\{\frac{1-e^{-\alpha}}{\alpha}, \ \frac{1-e^{-\alpha}}{1-e^{-2\alpha}}\right\}$.

We now determine the parameter α that maximizes this quantity $\gamma_a\gamma_o$. Note that the function $\frac{1-e^{-\alpha}}{\alpha}$ is a decreasing function of α, whereas $\frac{1-e^{-\alpha}}{1-e^{-2\alpha}}$ is an increasing function of α. Thus, in order to maximize their minimum, we should equalize the two terms, which is achieved at the unique positive value α^* satisfying $\alpha^* = 1 - e^{-2\alpha^*}$. This optimized parameter α^* can be expressed as

$$\alpha^* \triangleq 1 + \frac{W_0(-2e^{-2})}{2} \approx 0.7968, \tag{10}$$

where $W_0(.)$ is the principal branch of the Lambert W function (cf. Corless et al. 1996).

Under this choice of α^*, we get a competitive ratio lower bound of $\frac{1-e^{-\alpha^*}}{\alpha^*}$, which exceeds 0.68. To make the execution of this improved policy entirely concrete, we explicitly detail its operational steps in Algorithm 2. We then summarize our theoretical guarantees in the following theorem, which also establishes that the analysis for this algorithm $\pi^{\exp(\alpha^*)}$ is tight.

Algorithm 2 Optimal Exponential Index-Based Policy $\pi^{\mathsf{exp}(\alpha^*)}$

System Parameter: Set the optimized decay rate $\alpha^* \approx 0.7968$.

At any time t when a new request arrives (or an active request reaches its maximum lifespan):

1. **Expiration:** Remove any currently served request i that has reached its deadline (i.e., $t \geq t_i + 1$).

2. **Index Evaluation:** Let $\mathcal{P}(t)$ be the pool of candidate requests (the new arrival plus any remaining active requests). For every request $i \in \mathcal{P}(t)$, compute its instantaneous exponential priority score based on its intrinsic height h_i and arrival time t_i:

$$f_i(t) = h_i e^{-\alpha^*(t-t_i)}.$$

3. **Capacity Allocation:**

- If $|\mathcal{P}(t)| \leq C$, retain all requests in $\mathcal{P}(t)$.
- If $|\mathcal{P}(t)| > C$, retain the C requests in $\mathcal{P}(t)$ possessing the highest priority scores $f_i(t)$. Irreversibly preempt (or reject) the remaining $|\mathcal{P}(t)| - C$ requests.

THEOREM 2. *The competitive ratio $\mathcal{CR}$ of policy $\pi^{\mathsf{exp}(\alpha^*)}$ satisfies*

$$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) = \frac{1 - e^{-\alpha^*}}{\alpha^*} \geq 0.68.$$

5. Upper Bounds

In Sections 3 and 4, we considered specific index-based policies and provided tight analyses for their competitive ratios, under our core model. In this section we derive upper bounds on the competitive ratio of *any* algorithm, which provide some benchmarks for how close our algorithms are to theoretically optimal under our core model.

First, we show that the competitive ratio of the best algorithm we derived, the exponential-index-based policy with a competitive ratio exceeding 0.68, beats what can be obtained using the looser Skyline relaxation. In fact, even the optimal offline policy OPT cannot earn more reward than $\frac{2}{3}\mathsf{Skyline}$. This shows why we had to consider the more sophisticated behavior of OPT in Section 4 to beat 2/3 (compared to, e.g., the simpler Lemma 1 in Section 3).

PROPOSITION 3 (OPT **vs.** $\mathsf{Skyline}$). *The worst-case ratio of $\frac{\mathsf{OPT}(\mathcal{I})}{\mathsf{Skyline}(\mathcal{I})}$ among all $\mathcal{I}$ is upper bounded by $\frac{2}{3}$. This fact holds under any amount of capacity C.*

Next we show that even comparing to OPT, algorithms that are deterministic (which all of our index-based policies satisfy) cannot have a competitive ratio exceeding 0.84.

PROPOSITION 4 **(Deterministic Algorithms vs. OPT).** *When $C = 1$, the competitive ratio of any feasible deterministic online policy $\pi \in \Pi_f^{on}$ compared to optimal offline is upper bounded by*

$$\mathcal{CR}(\pi) \leq x^*(2 - x^*) \leq 0.84,$$

where $x^* \approx 0.597$ is the unique solution to $2x^2(2-x) = 1$ on $[0,1]$.

Finally, we briefly discuss randomized algorithms, which specify a distribution of deterministic algorithms to use. Their utility is defined as the expected utility of a deterministic algorithm drawn from this distribution. In the definition of competitive ratio, the worst-case instance $\mathcal{I}$ must be chosen without knowing the deterministic algorithm drawn (assuming an oblivious adversary, as standard), which is why randomized algorithms can sometimes achieve better competitive ratios than their deterministic counterparts. Nonetheless, we modify the construction from Proposition 4 to show that randomized algorithms cannot do too much better.

PROPOSITION 5 **(Randomized Algorithms vs. OPT).** *When $C = 1$, the competitive ratio of any feasible randomized online policy compared to optimal offline is upper bounded by*

$$\frac{1 + x^*(2 - x^*)}{2} \leq 0.92.$$

6. Extension: Heterogeneous Lifespans

In this section, we relax the assumption that all requests have the same lifespan. We assume that each request $i \in [N]$ has lifespan $\ell_i \in [1, L]$ for some $L \geq 1$, with L being known. We note that the lifespan feasibility constraint Equation (3) is now transformed to

$$\mathcal{H}(t) \bigcap \left\{ i \in [N] \middle| t_i + \ell_i \leq t \right\} = \varnothing.$$

One could naively apply the algorithms designed from the core model with homogeneous lifespans earlier, but this approach would lose a factor of L in the competitive ratio. In Subsection 6.1, we provide a more elaborate design, which enables any algorithm studied for the core model to handle heterogeneous lifespans while losing a factor of at most $\sqrt{L}$ in the competitive ratio. We also show this loss to be unavoidable; namely, the competitive ratio of any deterministic algorithm under heterogeneous lifespans is upper bounded by $1/\sqrt{L}$. Later in Subsection 6.2, we prove that the competitive ratio can be significantly improved to $\Omega\left(1/\log(L)\right)$ under a large-capacity regime where $C = \Omega\left(\log(L)\right)$.

6.1. Regime: General C

In this subsection, we do not make any assumptions on the capacity and allow it to be any $C \geq 1$. Let us start with the following definition of transforming an original instance $\mathcal{I}$ with heterogeneous lifespans into a transformed instances $\widetilde{\mathcal{I}}$ with homogeneous lifespans.

DEFINITION 7 ($\widetilde{\mathcal{I}}$**: Transformed Instance)**. Given an instance $\mathcal{I} = \big(N, \{(t_i, h_i, \ell_i)\}_{i\in[N]}\big)$, where all lifespans are in $[1, L]$, we construct the following transformed instance. For each request i,

1. if $\ell_i \geq \sqrt{L}$, we truncate its lifespan to $\widetilde{\ell}_i = \sqrt{L}$ and keep other parameters the same, i.e., $\widetilde{h}_i = h_i$ and $\widetilde{t}_i = t_i$;

2. if $\ell_i < \sqrt{L}$, we stretch the request to have lifespan $\sqrt{L}$ while preserving its total area and arrival time, i.e., $\widetilde{\ell}_i = \sqrt{L} > \ell_i$, $\widetilde{h}_i = \frac{h_i \ell_i}{\sqrt{L}} < h_i$, and $\widetilde{t}_i = t_i$.

We define $\widetilde{\mathcal{I}} \triangleq \Big(N, \{(\widetilde{t}_i, \widetilde{h}_i, \widetilde{\ell}_i)\}_{i\in[N]}\Big)$, and let $\widetilde{z}_i$ be the request in $\widetilde{\mathcal{I}}$ that corresponds to request z_i in $\mathcal{I}$ for all $i \in [N]$.

Note that all the request arrival times are the same between $\mathcal{I}$ and $\widetilde{\mathcal{I}}$. Whenever a request arrives in the original instance $\mathcal{I}$, since its lifespan is known at arrival, we can compute its corresponding request in $\widetilde{\mathcal{I}}$ and construct $\widetilde{\mathcal{I}}$ in real-time.

Now consider any policy $\widetilde{\pi}$ with competitive ratio $\mathcal{CR}(\widetilde{\pi})$ under the homogeneous lifespan setting where all lifespans are equal to $\sqrt{L}$. For simplicity, we can just think about $\widetilde{\pi} \in \big\{\pi^{\mathsf{h}}, \pi^{\mathsf{ra}}, \pi^{\mathsf{exp}(\alpha^*)}\big\}$ with known competitive ratios. Note that after scaling the $\sqrt{L}$ lifespan to 1, we get all the same competitive ratio results as before.

The intuition behind the construction in Definition 7 is that an algorithm can (easily) collect at least as much reward on the original instance $\mathcal{I}$ compared to the transformed instance $\widetilde{\mathcal{I}}$; meanwhile, the optimal reward on $\mathcal{I}$ compared to $\widetilde{\mathcal{I}}$ is greater by a factor of at most $\sqrt{L}$. Our proof is heavily based on this intuition. Now we formalize it, starting from the construction of an algorithm $\pi^{\mathsf{red}}(\widetilde{\pi})$ for heterogeneous lifespans based on the algorithm $\widetilde{\pi}$ for homogeneous lifespans.

DEFINITION 8 ($\pi^{\mathsf{red}}(\widetilde{\pi})$**: Reduction Policy)**. Given any heterogeneous instance $\mathcal{I}$ and a policy $\widetilde{\pi}$ with competitive ratio $\mathcal{CR}(\widetilde{\pi})$ under the homogeneous lifespan setting, let $\widetilde{\mathcal{I}}$ be the homogeneous transformed instance of $\mathcal{I}$ and we construct $\pi^{\mathsf{red}}(\widetilde{\pi})$ as follows. For any request i at its arrival time, we feed the arrival of $\widetilde{z}_i$ into $\widetilde{\pi}$ and specify its departure time under $\pi^{\mathsf{red}}(\widetilde{\pi})$ as: $d_i^{\pi^{\mathsf{red}}(\widetilde{\pi})} = \min(d_i^{\widetilde{\pi}}, t_i + \ell_i)$.

Note that the second argument $t_i + \ell_i$ in the min operator (in the definition of $d_i^{\pi^{\mathsf{red}}(\widetilde{\pi})}$) can only be binding if $\ell_i < \sqrt{L}$, in which case the lifespan of i in the transformed instance is longer than its original lifespan. In the rest of this section as well as its proofs in the appendix, we simply use $\widetilde{\mathcal{CR}} \triangleq \mathcal{CR}(\widetilde{\pi})$ and $\pi^{\mathsf{red}} \triangleq \pi^{\mathsf{red}}(\widetilde{\pi})$ for notational brevity.

Combining the way we determined the departure times under π^{red} with the fact that $\widetilde{\pi}$ is an online feasible policy that is running on $\widetilde{\mathcal{I}}$, we can conclude that: (i) π^{red} is also an online policy which never re-serves requests that it has preempted, (ii) π^{red} satisfies the capacity constraint, because we only reduce the departure times compared to $\widetilde{\pi}$, and (iii) π^{red} satisfies the lifespan constraint, because we ensure that the departure time for each request is no later than its deadline. In other words, we have $\pi^{\mathsf{red}} \in \Pi_f^{on}$, simply by construction.

We are now ready to state the main result of this subsection.

THEOREM 3. *For any instance $\mathcal{I}$ and policy $\widetilde{\pi}$ with competitive ratio of $\widetilde{\mathcal{CR}}$ under the homogeneous lifespan setting, we have*

$$U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}} \mathsf{OPT}(\mathcal{I}).$$

It follows that $\mathcal{CR}(\pi^{\mathsf{red}}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}}$.

We remark that the policy $\widetilde{\pi}$ can be any of the policies we introduced in the preceding. For instance, we can choose $\widetilde{\pi} = \pi^{\mathsf{exp}(\alpha^*)}$, which gives us $\widetilde{\mathcal{CR}} = \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68$.

The previous result provides a lower bound on the competitive ratio with a decay rate of $O\left(\frac{1}{\sqrt{L}}\right)$. We next show that this decay rate is the best that we can hope for, when $C = 1$.

PROPOSITION 6. *When $C = 1$, the competitive ratio of any deterministic feasible online policy $\pi \in \Pi_f^{on}$ compared to optimal offline is upper bounded by*

$$\mathcal{CR}(\pi) \leq \frac{1}{\sqrt{L}}.$$

6.2. Regime: $C = \Omega(\log(L))$

In this subsection, we show that if the capacity satisfies $C = \Omega(\log(L))$, then the competitive ratio can be improved dramatically from $\Omega\left(\frac{1}{\sqrt{L}}\right)$ to $\Omega\left(\frac{1}{\log(L)}\right)$. In order to show this, we divide an instance into multiple sub-instances and run a policy with a good competitive ratio on each of the sub-instances. The final policy is then the aggregate policy of all these sub-policies, each running on their corresponding sub-instance, which is constructed from the original instance in an online fashion.

Let us describe how to divide an instance $\mathcal{I}$ whose requests have lifespans in the range of $[1, L]$ into sub-instances $\{\mathcal{I}_k\}_{k\in[K]}$, for some $K \in [C]$ to be specified. $\mathcal{I}_k$ is the instance consisting of all requests in $\mathcal{I}$ whose lifespan lies in the interval $[L^{\frac{k-1}{K}}, L^{\frac{k}{K}}]$, for all $k \in [K] \triangleq \{1, 2, ..., K\}$. We define the capacity to be $\lfloor \frac{C}{K} \rfloor$ on each sub-instance. Notice that for each of these sub-instances, the ratio of the largest to the smallest possible lifespan is exactly $L^{\frac{1}{K}}$. For each $k \in [K]$, let π^k be a copy of the policy $\pi^{\mathsf{red}}(\widetilde{\pi})$ from Subsection 6.1, whose competitive ratio is at least $\frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}}$ (e.g., if $\widetilde{\pi} = \pi^{\mathsf{exp}(\alpha^*)}$, then $\widetilde{\mathcal{CR}} \geq 0.68$) for the sub-instance $\mathcal{I}_k$ whose requests possess lifespans only in the

interval $[L^{\frac{k-1}{K}}, L^{\frac{k}{K}}]$. Note that as requests in the instance $\mathcal{I}$ arrive to the system in real-time, we can route these requests to the sub-instances $\{\mathcal{I}_k\}_{k\in[K]}$ based on its lifespan length.

We now describe our proposed policy.

DEFINITION 9 (π_K^{div} **: Divider Policy)**. π_K^{div} has a total capacity of C and divides it into K categories, each consisting of $\lfloor \frac{C}{K} \rfloor$ units of capacity. As the requests in $\mathcal{I}$ arrive over time, for all $k \in [K]$, π_K^{div} constructs $\mathcal{I}_k$ in an online fashion, and with its k-th category of capacity (whose size is $\lfloor \frac{C}{K} \rfloor$), π_K^{div} emulates the exact behavior of running π^k on $\mathcal{I}_k$.

The next lemma provides a lower bound on the competitive ratio of policy π_K^{div}.

LEMMA 6. *For any instance $\mathcal{I}$ and $K \in [C]$, we have*

$$U_{\pi_K^{\mathsf{div}}}(\mathcal{I}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}).$$

It follows that $\mathcal{CR}\left(\pi_K^{\mathsf{div}}\right) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C}$.

We note that for each sub-instance, the optimum offline with $\lfloor \frac{C}{K} \rfloor$ capacity is at least $\lfloor \frac{C}{K} \rfloor / C$ times the optimum offline with C capacity. This explains the additional factor of $\lfloor \frac{C}{K} \rfloor / C$ in the competitive ratio. Intuitively, the worst case for this divider policy arises when all requests in the original instance $\mathcal{I}$ fall under the same sub-instance $\mathcal{I}_k$ and we have unnecessarily shrunk our capacity by a factor of K.

Now, let K^* be the choice of K that maximizes the lower bound provided in Lemma 6, i.e.,

$$K^* \triangleq \arg\max_{K\in[C]} \left\{ \frac{\lfloor \frac{C}{K} \rfloor}{\sqrt{L^{\frac{1}{K}}}} \right\}.$$

We next state the main theorem of this subsection: the competitive ratio of $\pi_{K^*}^{\mathsf{div}}$ is significantly improved under a large-capacity regime.

THEOREM 4. *Suppose $C = \Omega(\log(L))$. Then, we have*

$$\mathcal{CR}\left(\pi_{K^*}^{\mathsf{div}}\right) = \Omega\left(\frac{1}{\log(L)}\right).$$

7. Extension: Time-Varying Reward Rates

In this section, we relax the assumption of constant reward rates. This extension is most naturally motivated by settings in which the reward rate varies over time - notably order consolidation, where the rate reflects a customer's (typically decaying) likelihood of placing a follow-up order over the holding window, and adaptive LLM inference, where it reflects the diminishing marginal quality from additional inference time. We keep these applications in mind throughout the section; the homogeneity assumption introduced below is especially natural in the order-consolidation setting,

where customers may differ in their initial propensity to re-order while that propensity evolves in a common way over time.

Recall that in the core model, the reward collected from request i by algorithm π is equal to $h_i \times (d_i^\pi - t_i)$, where the reward rate of request i remained constant and equal to h_i over the entire time interval $[t_i, d_i^\pi)$. Now we relax this assumption and allow the reward rate to vary over time, i.e., the reward rate of request i at time $t \geq t_i$ is $\mathfrak{h}_i(t)$, where $\mathfrak{h}_i(\cdot)$ is a mapping from $\mathbb{R}_+$ to $\mathbb{R}_+$. This function $\mathfrak{h}_i(t)$ depends on request i and time t, and can be regarded, in the order-consolidation setting, as the derivative of the expected number of orders from customer i during time interval $(t_i, t]$ w.r.t. t. In order to incorporate this change in the definitions of $U_\pi(\mathcal{I})$ and $\mathsf{OPT}(\mathcal{I})$, as well as all of the definitions and results in Section 4, whenever we calculate the reward of some request $z_i = (t_i, \mathfrak{h}_i(.))$ up to some time t', we replace $h_i \times (t' - t_i)$ by $\int_{t_i}^{t'} \mathfrak{h}_i(t'')\, dt''$. Lastly, r_z was defined in Section 4.1 as the index of the base curve that passes through the point (t, h), i.e., the unique r such that $h = \mathcal{B}_r(t)$, for a generic arrival $z = (t, h)$. In this extension, a generic arrival is denoted by $z = (t, \mathfrak{h}(.))$; hence, we change the definition of r_z to be the index of the unique base curve that passes through the point $(t, \mathfrak{h}(t))$, i.e., the unique r such that $\mathfrak{h}(t) = \mathcal{B}_r(t)$. With the integration of these changes, we emphasize that all of our results in Subsection 4.2 with similar proofs still hold, thereby making Theorem 1 to hold even under this extension.

If no conditions are imposed on $\mathfrak{h}_i(t)$, then the competitive ratio can be arbitrarily small, as such reward functions can drop to 0 at any time and capture arbitrarily heterogeneous lifespans as discussed in Section 6. Therefore, we make the following "homogeneity" assumption throughout this section, which says that the reward of request i over time t can be decomposed as the product of two terms:

$$\mathfrak{h}_i(t) = \mathfrak{h}_i(t_i) \cdot \mathfrak{r}(t - t_i) \quad \forall t \geq t_i. \tag{11}$$

Here, $\mathfrak{h}_i(t_i)$ represents the initial value of the reward that depends on i, and $\mathfrak{r} : \mathbb{R}_+ \to \mathbb{R}_+$ satisfying $\mathfrak{r}(0) = 1$ captures the decay rate over time (although we call $\mathfrak{r}$ the decay rate function, we also allow its value to become greater than one, capturing settings in which the reward rate increases over time, such as a customer's likelihood of a follow-up order rising). It is important to note that this decay rate function $\mathfrak{r}$ is homogeneous across all requests, and it takes as input only the elapsed time $t - t_i$ of request i.

This underlying homogeneity assumption is reasonable whenever all requests share a common temporal reward profile—for instance, when customers have the same decay rate in their likelihood of a subsequent order over time. In other words, requests may differ in their initial reward rate even though that rate evolves identically over time across requests (e.g., customers may differ in their initial inclination to reorder, while their propensity to do so changes uniformly over time).

Nevertheless, it worth mentioning that if we allow varying decay rates across customers in addition to their different initial likelihood of multiorder, the model would become excessively intricate. This would lead to an overwhelming number of parameters that need to be learned, potentially hindering the effectiveness and practicality of the model.

We focus on the exponential base $\mathcal{B}^{\exp(\alpha)}$ with $\mathcal{B}_r^{\exp(\alpha)}(t) = re^{-\alpha t}$ as defined in (9), to take advantage of the proof techniques we have developed to "analytically" compute the associated γ_a and γ_o. Similar to (6) and (7), we define

$$\gamma_o \triangleq \min_{z\triangleq(t,\mathfrak{h}(.)),\ t'\in[t,t+1)} \left\{ \frac{\int_t^{t'} \mathcal{B}_{r_z}^{\exp(\alpha)}(t'')\ dt''}{\int_t^{t'} \mathfrak{h}(t'')\ dt''} \right\},$$

$$\gamma_a \triangleq \min_{z\triangleq(t,\mathfrak{h}(.))} \left\{ \min_{t'\in[t,t+1)} \left\{ \frac{\int_t^{t'} \mathfrak{h}(t'')\ dt''}{\int_t^{t'} \mathcal{B}_{r_z}^{\exp(\alpha)}(t'')\ dt''} \right\},\ \frac{\int_t^{t+1} \mathfrak{h}(t'')\ dt''}{\int_t^{t+2} \mathcal{B}_{r_z}^{\exp(\alpha)}(t'')\ dt''} \right\}.$$

Under Assumption (11), it is fairly easy to compute the values of γ_o and γ_a. Define

$$R \triangleq \int_0^1 \mathfrak{r}(t'')\ dt'' > 0. \tag{12}$$

Lemma 7. *Under Assumption* (11)*, for any* $\alpha \in \mathbb{R}$*, we have*

$$\gamma_o = \min_{t,\ t'\in[t,t+1)} \left\{ \frac{\int_0^{t'-t} e^{-\alpha t''}\ dt''}{\int_0^{t'-t} \mathfrak{r}(t'')\ dt''} \right\} \leq \frac{1-e^{-\alpha}}{\alpha R}, \tag{13}$$

$$\gamma_a = \min\left\{ \min_{t,\ t'\in[t,t+1)} \left\{ \frac{\int_0^{t'-t} \mathfrak{r}(t'')\ dt''}{\int_0^{t'-t} e^{-\alpha t''}\ dt''} \right\},\ \frac{\alpha R}{1-e^{-2\alpha}} \right\} \leq \min\left\{1,\ \frac{\alpha R}{1-e^{-2\alpha}}\right\}. \tag{14}$$

In addition, both the inequalities in (13) *and* (14) *will become equality if the function* $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ *is non-increasing in* t'' *on* $[0,1]$.

If $\alpha = 0$, then the expressions involving α in (13) and (14) are understood to denote their limits as $\alpha \to 0$.

As our goal is to maximize $\gamma_a\gamma_o$, Lemma 7 provides a sufficient condition on when the upper bound on $\gamma_a\gamma_o$ can be achieved. Therefore, combining Theorem 1 with the assumption that $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ is non-increasing in t'' on $[0,1]$, we can conclude that

$$\mathcal{CR}(\pi^{\exp(\alpha)}) \geq \gamma_a\gamma_o = \min\left\{ \frac{1-e^{-\alpha}}{\alpha R},\ \frac{1-e^{-\alpha}}{1-e^{-2\alpha}} \right\}, \tag{15}$$

where $\pi^{\exp(\alpha)}$ is the index-based policy with exponential base $\mathcal{B}^{\exp(\alpha)}$ as defined in Section 4.

Now we want to maximize the value of (15) over α. Note that the first term in the min operator is decreasing in α, and the second term is increasing in α. As such, the maximum value is achieved when the two terms are equal to each other, i.e.,

$$R = \begin{cases} \frac{1-e^{-2\alpha}}{\alpha} & \text{when } \alpha \neq 0; \\ 2 & \text{when } \alpha = 0. \end{cases} \tag{16}$$

We set $\alpha = \alpha^*(R)$ as the unique non-trivial real solution to the equation $\alpha R = 1 - e^{-2\alpha}$ for any $R \neq 2$ (as 0 is always a trivial solution), and set $\alpha = \alpha^*(R) = 0$ for $R = 2$. Equivalently,

$$\alpha^*(R) \triangleq \frac{W_\gamma\left(-\frac{2e^{-\frac{2}{R}}}{R}\right)}{2} + \frac{1}{R}, \quad \gamma \triangleq \begin{cases} 0 & \text{if } R \leq 2; \\ -1 & \text{if } R > 2, \end{cases} \tag{17}$$

where W is the Lambert W function and γ indicates which branch of the Lambert W function to be used (cf. Corless et al. 1996). Note that

$$\alpha^*(R) > 0 \quad \text{ if and only if } \quad R < 2,$$

which comes from the fact that the function $\frac{1-e^{-2\alpha}}{\alpha}$ is decreasing in α and reaches 2 at $\alpha = 0$ (more precisely, as $\alpha \to 0$). Thus, we use the principal branch of the Lambert W function if $R \leq 2$, and use the -1 branch otherwise. See an illustration of $\alpha^*(R)$ as a function of R in Figure 3. It follows that

$$\gamma_a = 1, \qquad \gamma_o = \frac{1 - e^{-\alpha^*(R)}}{\alpha^*(R)R} = \frac{1 - e^{-\alpha^*(R)}}{1 - e^{-2\alpha^*(R)}} = \frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)} + 1}. \tag{18}$$

It is important to know that to achieve the values of (18), we only require the function $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ to be non-increasing for the value $\alpha = \alpha^*(R)$, not all $\alpha \in \mathbb{R}$.

We now discuss the critical assumption (aside from homogeneity) that $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$ is non-increasing over $t'' \in [0,1]$. If $\mathfrak{r}$ is differentiable with derivative $\mathfrak{r}'$, then this condition is equivalent to

$$\mathfrak{r}'(t'') + \alpha^*(R)\mathfrak{r}(t'') \geq 0 \quad \forall t'' \in [0,1]. \tag{19}$$

It can then be checked that (19) is satisfied by an exponential function $\mathfrak{r}(t) = e^{-\beta t}$ for any $\beta \in \mathbb{R}$, due to the fact that $\alpha^*(R) > \beta$ (via the definition of $\alpha^*(R)$ in (17) that satisfies (16) and the calculation that $R = \frac{1-e^{-\beta}}{\beta}$ in (12)). This allows us to model, for instance, decaying customer multi-order probabilities or diminishing marginal quality improvements in LLM inference by setting $\beta > 0$. Note that it reduces to the core model with constant reward rates when $\beta = 0$.

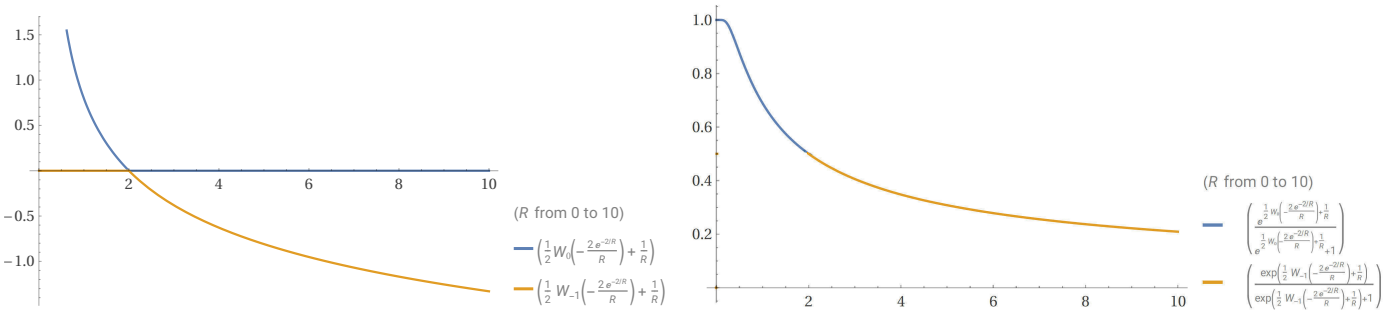
We summarize our findings of this section in the next theorem. Its proof follows similarly from the proof of Theorem 2 and the calculations of γ_a, γ_o as discussed above.

THEOREM 5. *Under homogeneous time-varying reward rates given by a function $\mathfrak{r}$ for which $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$ is non-increasing over $t'' \in [0,1]$, with R and $\alpha^*(R)$ as defined in* (12) *and* (17)*, respectively, we have*

$$\mathcal{CR}\left(\pi^{\mathsf{exp}(\alpha^*(R))}\right) = \gamma_a\gamma_o = \frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)} + 1}.$$

See an illustration of $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as a function of R in Figure 3. In particular, when $R=1$, we recover the results in Section 4.3 that $\alpha^*(1)\approx 0.7968$ and $\mathcal{CR}\left(\pi^{\exp(\alpha^*(1))}\right)\geq 0.68$. Moreover, $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ is decreasing in R, starting from 1 when $R=0$. This is because R can be regarded as a parameter for measuring "risk." Indeed, the greater the value of R, the higher the potential regret could be for preempting a request too early.

As a final note, algorithm $\pi^{\exp(\alpha^*(R))}$ only requires the knowledge of R and the initial reward value $\mathfrak{h}_i(t_i)$ of each request, and the guarantee is robust to the shape of the reward function $\mathfrak{r}(.)$ itself (as long as it satisfies the non-increasing property for $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$).



(a) $\alpha^*(R)$ as a function of R

(b) $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as a function of R

Figure 3 **Illustrations of $\alpha^*(R)$ and $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as functions of R. In Figure (a), the value of $\alpha^*(R)$ follows the blue curve when $R\leq 2$, and follows the orange curve when $R>2$.**

8. Numerical Experiments

In this section, we conduct a concise numerical study to illustrate the average-case behavior of the three algorithms (π^{h}, π^{ra} and $\pi^{\exp(\alpha)}$) under the core model with constant reward rates and homogeneous lifespans. These experiments are meant to show how the policies behave under simple stochastic (Poisson) arrival conditions; they complement, rather than substitute for, our worst-case analysis. We further note that, because all of our competitive-ratio analyses are tight, each policy provably attains its worst-case ratio on the explicit adversarial instances constructed in the corresponding proofs; the study below instead probes the more benign, typical-case regime.[6]

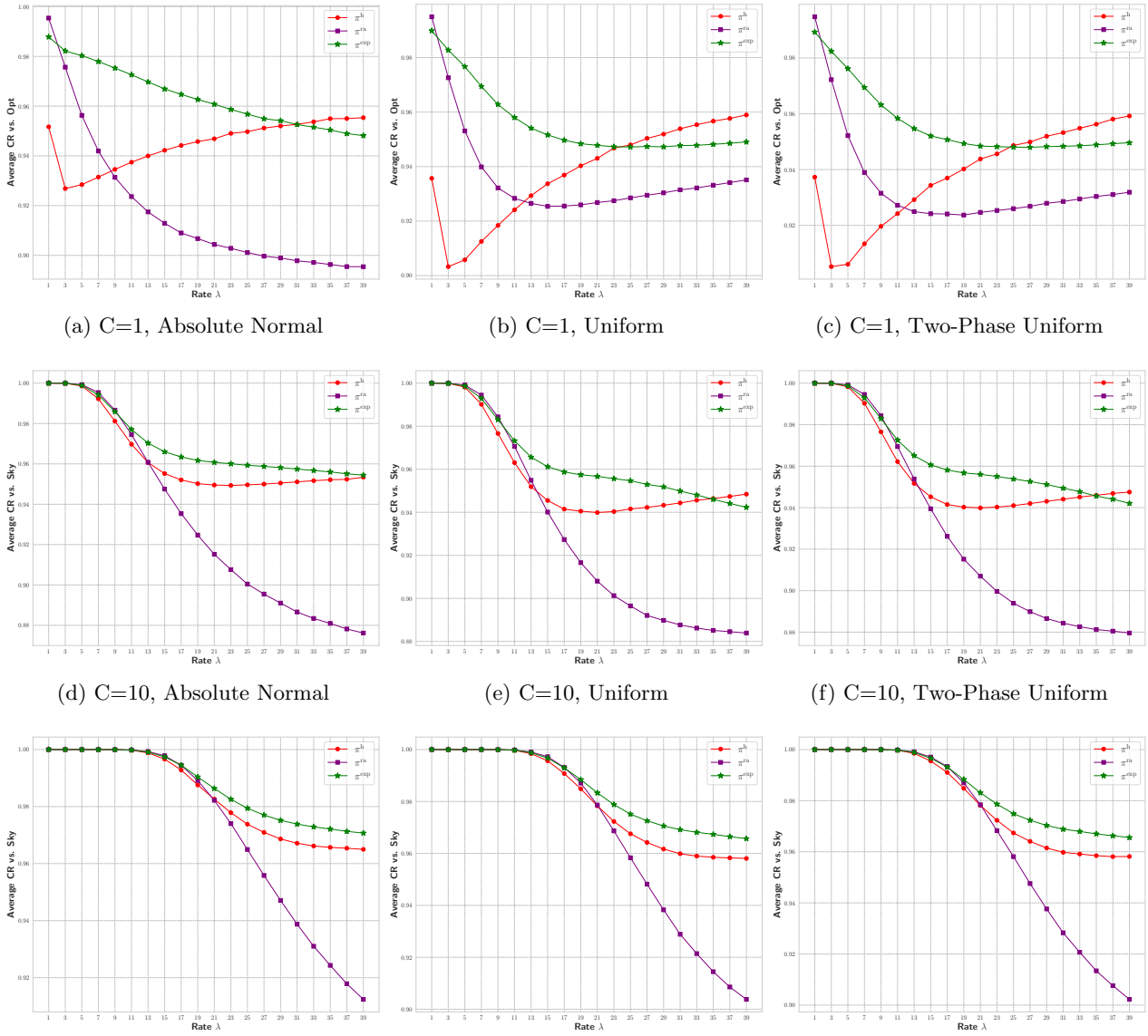
Experimental Setup. We pick capacity $C\in\{1,10,20\}$ and fix the time horizon of the problem T and generate $M=1,000$ instances of an arrival sequence of requests over time from a Poisson process with rate λ. For each request, its reward rate (height) is generated independently from a distribution. We consider three types of distributions: a uniform distribution on [0,1], the absolute value of a standard normal distribution, and a two-phase distribution in which the first half of the

[6] The code and instructions to reproduce all figures in this section are publicly available at `https://github.com/MR-Aminian/Online_Job_Selection_Git`.

heights follows from a uniform distribution on [0,1] and the second half follows from a uniform distribution on [0,0.5]. Here, the two-phase distribution, with decreasing means, amplifies the challenge in deciding whether to switch to a newly-arrived request with lower height. Note that as C grows, the computational complexity of all of our index-based policies grows linearly (keeping them suitable for practice), whereas the size of the state-space grows exponentially, hindering the ability to compute the optimum offline. In this regard, when $C = 1$, we set $T = 50$ and consider the true optimum offline (OPT) as the performance metric. However, for $C \in \{10, 20\}$, we set $T = 20$ and consider the Skyline relaxation ($\mathsf{Skyline}$) as the performance metric, which is a stronger benchmark with higher value. Regarding performance metric, we use the average (over M instances) of the performance ratio against the corresponding benchmark under the same instance. Under each of the three height distributions, we compute these average performance ratios - for all of the three policies - across various values of λ, and plot the ratio as a function of λ. As shown in Figure 4, the results comply with our expectations formed from the theoretical worst-case ratio analysis for each of the three policies. We now discuss the main findings/takeaways from the numerical results.

Takeaways from Experimental Results. The average ratios of the three algorithms (against either optimum offline or Skyline) - are much higher compared with their theoretic lower bounds, indicating the fact that these simple and interpretable algorithms also perform well in non-adversarial settings. The policy π^{h} is most effective in scenarios where the rate of arrivals is much faster than the capacity to serve requests, because it maximizes the reward rate from each unit of capacity. Conversely, the policy π^{ra} excels in situations where arrivals are significantly slower compared to the capacity, except in cases where the arrival rate is so slow that all policies yield optimal results. In scenarios of moderate arrival rates, the exponential-index-based policy $\pi^{\mathsf{exp}(\alpha)}$ derived from our worst-case analysis framework outperforms others, even in average-case scenarios.

Explanations of Experimental Results. When $C = 1$, we first observe a sharp decrease in the performance ratio of π^{h} around $\lambda = 3$, which recovers shortly after. The explanation behind this phenomenon is that when $\lambda \ll 1$ the system is empty enough so that the capacity is not binding and all the three algorithms essentially serve requests until deadlines. Their ratios are close to one. When λ becomes around 3 (meaning around 3 arrivals per unit of time), then it becomes possible that π^{h} rejects a fresh request whose height is slightly lower than the currently served (old) request, leading to a sharp decrease on its performance. However, as λ increases, the likelihood of an incoming request having a greater height also rises. Consequently, both π^{h} and OPT start favoring these newer requests, surpassing the likelihood of scenarios where only the optimal offline algorithm makes the switch. This shift leads to a gradual improvement in the performance of π^{h}, albeit in a moderate manner.



(a) C=1, Absolute Normal (b) C=1, Uniform (c) C=1, Two-Phase Uniform

(d) C=10, Absolute Normal (e) C=10, Uniform (f) C=10, Two-Phase Uniform

(g) C=20, Absolute Normal (h) C=20, Uniform (i) C=20, Two-Phase Uniform

Figure 4 Average ratios against Optimal Offline (a)-(c) or Skyline (d)-(i) as a function of arrival rate λ.

A nuanced interpretation can be provided for π^{ra} as well. When the distribution of height follows Absolute Normal in which heights tend to cluster around zero, we can notice that the performance of π^{ra} continues to deteriorate as λ increases. This decline is rooted in the fact that, unlike π^{h} whose worst-case scenario involves only two requests (see the proof of Proposition 1), π^{ra} grapples with infinitely many arrivals in its worst-case instance (see the proof of Proposition 2). Consequently, the more frequent the occurrence of extended sequences of decreasing requests, the closer we get to this worst-case scenario. By raising λ, we increase the likelihood of detrimental switches under π^{ra} for these long decreasing sequences, ultimately diminishing π^{ra}'s performance. However, when the

distribution of heights is not heavily concentrated around zero, such as in the Uniform distribution, the probability of encountering long decreasing sequences becomes much smaller than under the Absolute Normal distribution. This makes it less likely for π^{ra} to follow a similar path as its worst-case instance. Furthermore, the likelihood of encountering increasing sequences grows, resulting in more instances where π^{ra} and OPT make the same decisions. Taking these two effects into account, under the Uniform or Two-phase Uniform distributions, we observe that the decreasing trend in π^{ra} diminishes, and even reverses, more swiftly compared to the Absolute Normal distribution.

Regarding $\pi^{\mathsf{exp}(\alpha)}$, since it is more balanced than the other two, its performance ratio remains stable when facing two different styles of worst-case instances as described above (one with two arrivals for π^{h} and one with infinitely many arrivals for π^{ra}). The balancing nature implies that the rate of performance change w.r.t. λ for $\pi^{\mathsf{exp}(\alpha)}$ should fall somewhere between that of π^{ra} and π^{h}. This intuition is confirmed in our numerical results, because the increasing and decreasing trends as discussed above seem to offset each other when applied to $\pi^{\mathsf{exp}(\alpha)}$, which ensures that $\pi^{\mathsf{exp}(\alpha)}$ maintains a consistent performance regardless of values in λ.

In regard to the effect of capacity C, one might initially think that scaling both C and λ by the same factor would lead to minimal changes in the average performance ratios. Surprisingly, our experiments show otherwise, as evident from the plots. It appears that this relationship is more additive than multiplicative. This means that increasing both C and λ by the same amount results in roughly unchanged average performance ratios.

Lastly, we observe that a transition in the distribution has a minimal effect. This is evidenced by the high similarity in results for uniform versus two-phase uniform distributions. The results suggest that variations in distribution have only a small influence on the overall outcomes of our experiment.

9. Conclusion

In this paper, we introduced a new online decision-making problem where one must decide which requests to serve under a capacity constraint, facing a fundamental keep/switch trade-off. We demonstrated that this abstraction captures key features of several high-stakes operational settings, including cloud spot markets, adaptive LLM inference, order consolidation, and contingent workforce management. Methodologically, we introduced a novel framework based on "parallel partitioning" and "quasi-utility" that decouples the true utilities of online and offline policies while coupling their quasi-utilities, enabling the derivation of sharp constant-factor competitive ratio guarantees. We proved tightness results where applicable and extended the analysis to general settings with heterogeneous lifespans and time-varying reward rates. From an implementation standpoint, the resulting index-based algorithms are simple, transparent in balancing reward rate

against remaining opportunity, robust to demand uncertainty, and—as shown in our numerical study—achieve empirical performance significantly exceeding their worst-case theoretical bounds.

References

Abels A, Pitschmann E, Schmand D (2023) Prophet inequalities over time. *Proceedings of the 24th ACM Conference on Economics and Computation*, 1–20, EC '23 (New York, NY, USA: Association for Computing Machinery), ISBN 9798400701047, URL `http://dx.doi.org/10.1145/3580507.3597741`.

Ata B, Lee D, Sönmez E (2019) Dynamic volunteer staffing in multicrop gleaning operations. *Operations Research* 67(2):295–314.

Baek J, Ma W (2022) Bifurcating constraints to improve approximation ratios for network revenue management with reusable resources. *Operations Research* 70(4):2226–2236.

Besbes O, Elmachtoub AN, Sun Y (2019) Static pricing: Universal guarantees for reusable resources. *Proceedings of the 2019 ACM Conference on Economics and Computation*, 393–394.

Boysen N, Stephan K, Weidinger F (2019) Manual order consolidation with put walls: the batched order bin sequencing problem. *EURO Journal on Transportation and Logistics* 8(2):169–193.

Buchbinder N, Kimbrel T, Levi R, Makarychev K, Sviridenko M (2013) Online make-to-order joint replenishment model: Primal-dual competitive algorithms. *Operations Research* 61(4):1014–1029.

Canetti R, Irani S (1995) Bounding the power of preemption in randomized scheduling. *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing*, 606–615, STOC '95 (New York, NY, USA: Association for Computing Machinery), ISBN 0897917189, URL `http://dx.doi.org/10.1145/225058.225278`.

Charikar M, Henzinger M, Nguyen HL (2014) Online bipartite matching with decomposable weights. *European Symposium on Algorithms*, 260–271 (Springer).

Chen S, Yan Z, Lim YF (2024) Managing the personalized order-holding problem in online retailing. *Manufacturing & Service Operations Management* 26(1):47–65.

Chrobak M, Epstein L, Noga J, Sgall J, van Stee R, Tichỳ T, Vakhania N (2003) Preemptive scheduling in overloaded systems. *Journal of Computer and System Sciences* 67(1):183–197, ISSN 0022-0000.

Chrobak M, Jawor W, Sgall J, Tichý T (2007) Online scheduling of equal-length jobs: Randomization and restarts help. *SIAM Journal on Computing* 36(6):1709–1728.

Corless RM, Gonnet GH, Hare DE, Jeffrey DJ, Knuth DE (1996) On the lambert w function. *Advances in Computational mathematics* 5:329–359.

Dasgupta A, Ghosh A, Nazerzadeh H, Raghavan P (2009) Online story scheduling in web advertising. *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1275–1284, SODA '09 (USA: Society for Industrial and Applied Mathematics).

Devanur NR, Jain K, Kleinberg RD (2013) *Randomized Primal-Dual Analysis of RANKING for Online Bipartite Matching*, 101–107 (SIAM), URL http://dx.doi.org/10.1137/1.9781611973105.7.

Dickerson JP, Sankararaman KA, Srinivasan A, Xu P (2021) Allocation problems in ride-sharing platforms: Online matching with offline reusable resources. *ACM Transactions on Economics and Computation (TEAC)* 9(3):1–17.

Ekbatani F, Feng Y, Kash I, Niazadeh R (2024) Online job assignment. *SSRN Electronic Journal* URL https://api.semanticscholar.org/CorpusID:269100314.

Epstein L, Jeż L, Sgall J, Stee R (2016) Online scheduling of jobs with fixed start times on related machines. *Algorithmica* 74(1):156–176, ISSN 0178-4617, URL http://dx.doi.org/10.1007/s00453-014-9940-2.

Feng Y, Caldentey R, Xin L, Zhong Y, Wang B, Hu H (2024) Designing sparse graphs for stochastic matching with an application to middle-mile transportation management. *Management Science* 70(12):8988–9013.

Feng Y, Niazadeh R, Saberi A (2022) Near-optimal bayesian online assortment of reusable resources. *Proceedings of the 23rd ACM Conference on Economics and Computation*, 964–965.

Gao Z, Ling Z, Gupta V, Xin L (2022) Real-time omnichannel fulfillment optimization, available at SSRN: https://ssrn.com/abstract=4314250.

Golrezaei N, Nazerzadeh H, Rusmevichientong P (2014) Real-time optimization of personalized assortments. *Management Science* 60(6):1532–1551.

Golrezaei N, Yao E (2025) Upfront commitment in online resource allocation with patient customers. *Management Science* Forthcoming.

Gong XY, Goyal V, Iyengar GN, Simchi-Levi D, Udwani R, Wang S (2022) Online assortment optimization with reusable resources. *Management Science* 68(7):4772–4785.

Goyal V, Iyengar G, Udwani R (2020) Online allocation of reusable resources via algorithms guided by fluid approximations. *arXiv preprint arXiv:2010.03983* .

Huo T, Cheung WC (2022) Online reusable resource allocations with multi-class arrivals. *Available at SSRN 4320423* .

Kolen AW, Lenstra JK, Papadimitriou CH, Spieksma FC (2007) Interval scheduling: A survey. *Naval Research Logistics (NRL)* 54(5):530–543, URL http://dx.doi.org/10.1002/nav.20231.

Lei Y, Jasin S (2020) Real-time dynamic pricing for revenue management with reusable resources, advance reservation, and deterministic service time requirements. *Operations Research* 68(3):676–685.

Manshadi V, Rodilitz S (2020) Online policies for efficient volunteer crowdsourcing. *Proceedings of the 21st ACM Conference on Economics and Computation*, 315–316.

Mehta A, Saberi A, Vazirani U, Vazirani V (2007) Adwords and generalized online matching. *Journal of the ACM (JACM)* 54(5):22–es.

Neubert G, Adjadj P (2009) Capacity adjustment through contingent staffing outsourcing. *IFIP International Conference on Advances in Production Management Systems*, 195–202 (Springer).

Pauley E (2023) Farewell to the era of cheap ec2 spot instances. URL `https://pauley.me/post/2023/spot-price-trends/`.

Qin H, Xiao J, Ge D, Xin L, Gao J, He S, Hu H, Carlsson JG (2022) Jd. com: Operations research algorithms drive intelligent warehouse robots to work. *INFORMS Journal on Applied Analytics* 52(1):42–55.

Rusmevichientong P, Sumida M, Topaloglu H (2020) Dynamic assortment optimization for reusable products with random usage durations. *Management Science* 66(7):2820–2844.

Simchi-Levi D, Zheng Z, Zhu F (2025) On greedy-like policies in online matching with reusable network resources and decaying rewards. *Management Science* 71(10):8908–8926.

Wang Y, Wang T, Wang X, Deng Y, Cao L (2024) Data-driven order fulfillment consolidation for online grocery retailing. *INFORMS Journal on Applied Analytics* 54(3):211–221.

Wei L, Jasin S, Kapuscinski R (2021) Shipping consolidation across two warehouses with delivery deadline and expedited options for e-commerce and omni-channel retailers. *Manufacturing & Service Operations Management* 23(6):1634–1650.

Woeginger GJ (1994) On-line scheduling of jobs with fixed start and end times. *Theor. Comput. Sci.* 130(1):5–16, ISSN 0304-3975, URL `http://dx.doi.org/10.1016/0304-3975(94)90150-3`.

Wu D (2018) Going the extra mile in customer service: How does jd.com make order cancellation so easy?, jD Technology Blog.

Xie Y, Ma W, Xin L (2024) The benefits of delay to online decision-making. *Management Science* Forthcoming.

Yang J, Xin L, Zhong Y (2025) Losing sales to gain: Mitigating demand spillover in inventory networks. *Available at SSRN 5291958* .

Zhao Y, Wang X, Xin L (2025) Multi-item online order fulfillment in a two-layer network. *Operations Research* 73(5):2297–2305.

This page is intentionally blank. Proper e-companion title page, with INFORMS branding and exact metadata of the main paper, will be produced by the INFORMS office when the issue is being assembled.

E-Companion

Appendix A: Stochastic Reward Processes

Recall that in the main paper, we assume the reward process is deterministic, with the reward rate $h(t)$ being either time-invariant (core model) or time-varying (extension in Section 7). In this section, we demonstrate that a stochastic reward process can be reformulated as an equivalent deterministic reward process, provided it admits a deterministic compensator. This extension is particularly relevant in applications such as e-commerce order consolidation, where the multi-order cost-saving process for each customer is a stochastic jump process, accruing discrete rewards only when subsequent orders for an existing customer's order arrive.

Specifically, suppose that customer i's accumulated reward process is governed by a jump process $X_i(t)$, which is assumed to be càdlàg (right-continuous with left limits). Let $\mathbb{F}_i = \{\mathcal{F}_{i,t}\}_{t\geq 0}$ denote the natural filtration generated by $X_i(t)$, augmented to satisfy the usual conditions of completeness and right-continuity. To capture the strict non-anticipative online nature of the system, we define the standard global filtration $\mathbb{F} = \{\mathcal{F}_t\}_{t\geq 0}$, where $\mathcal{F}_t \triangleq \sigma\left(\bigcup_{\{i:t_i\leq t\}} \mathcal{F}_{i,t}\right)$ is the σ-algebra generated exclusively by the filtrations of customers who have arrived by time t.

Let $\lambda_i(t)$ denote the arrival intensity of the multi-order process at time t. To ensure a deterministic compensator, we assume $\lambda_i(t)$ is a deterministic càdlàg function, where $\lambda_i(t) = 0$ for all $t < t_i$. Let $w_i(t)$ represent the associated savings per multi-order arrival, drawn i.i.d. from a random variable W_i. The process $X_i(t)$ represents the accumulated reward for customer i up to time t, with $X_i(t) = 0$ for $t < t_i$. We define the deterministic reward rate as $h_i(t) \triangleq \lambda_i(t)\mathbb{E}[W_i]$. Consequently, the integral $\int_0^t h_i(s)ds$ serves as the deterministic compensator for $X_i(t)$, capturing the expected savings accumulated up to time t.

It follows that the compensated process

$$M_i(t) \triangleq X_i(t) - \int_0^t h_i(s)ds$$

is a zero-mean càdlàg martingale with respect to $\mathbb{F}_i$. Because the customers' arrival and reward processes are independent, $M_i(t)$ is also a zero-mean martingale with respect to the standard global filtration $\mathbb{F}$.

The dispatch time $\tau_i \geq t_i$ for order i under any standard non-anticipative online policy is thus a valid stopping time with respect to $\mathbb{F}$. Furthermore, because any order i must be dispatched before its strict deadline (lifespan), the stopping time τ_i is almost surely bounded. By applying Doob's Optional Stopping Theorem to the martingale $M_i(t)$ and the bounded stopping time τ_i, we obtain $\mathbb{E}[M_i(\tau_i)] = \mathbb{E}[M_i(0)] = 0$, which yields

$$\mathbb{E}[X_i(\tau_i)] = \mathbb{E}\left[\int_0^{\tau_i} h_i(s)ds\right] = \mathbb{E}\left[\int_{t_i}^{\tau_i} h_i(s)ds\right].$$

Here, $\mathbb{E}[X_i(\tau_i)]$ represents the expected cost savings physically accumulated through the stochastic multi-order consolidation, while $\mathbb{E}\left[\int_{t_i}^{\tau_i} h_i(s)ds\right]$ denotes the expected value of the accumulated deterministic reward rate $h_i(t)$ up to the dispatch time τ_i. Under this martingale property, the expected actual reward of any online algorithm (including ours) operating under $\mathbb{F}$ is perfectly equivalent to the reward accrued by operating on the deterministic reward-rate functions $h_i(t)$.

For problems governed by a stochastic reward process, we benchmark our policies against a clairvoyant optimal online algorithm. In contrast to the standard online policy, this clairvoyant benchmark operates

under an enlarged global filtration $\mathbb{F}^{\text{clair}} = \{\mathcal{F}_t^{\text{clair}}\}_{t\geq 0}$, formally defined as $\mathcal{F}_t^{\text{clair}} \triangleq \sigma\left(\bigcup_i \mathcal{F}_{i,t}\right)$. By taking the union over all customers i across the entire time horizon, this filtration explicitly provides the exact offline knowledge of all customer arrivals (timing and lifespans) in advance, while the exact multi-order jump realizations $\mathcal{F}_{i,t}$ are still only observed sequentially in time (see, e.g., Golrezaei et al. 2014). Because the individual jump processes are independent of the arrival sequence, $M_i(t)$ remains a valid martingale under $\mathbb{F}^{\text{clair}}$, and the clairvoyant dispatch decisions remain strictly defined as stopping times under $\mathbb{F}^{\text{clair}}$. Consequently, the clairvoyant's expected reward cleanly reduces to that of the optimal offline benchmark (OPT) in our deterministic model, which knows all customers i in the entire time horizon offline, as introduced in Subsection 3.1.

Finally, we note that this abstraction captures a broad class of stochastic behaviors, most notably compound non-homogeneous Poisson processes. In such settings, an individual customer's multi-order arrivals follow a non-homogeneous Poisson process, and the benefit from each realization is an i.i.d. random variable. In its simplest form, this reduces to a standard Poisson process with a constant arrival rate and a fixed jump size representing the cost saved per consolidated order. In conclusion, any stochastic reward process in our setting that admits a deterministic compensator can be rigorously reformulated as an equivalent deterministic reward process with rates $h_i(t)$, justifying the parsimonious reward rate assumptions utilized in our core model.

Appendix B: Extension: Heterogeneous Resources and Soft Capacity Constraints

In this section, we generalize our core framework to accommodate heterogeneous resources and "soft" capacity constraints. This extension captures environments where the system does not face a hard cliff of identical resources, but instead manages a hierarchy of processing tiers (e.g., varying GPU tiers for adaptive LLM inference, or varying virtual machines in cloud computing spot market).

B.1. Generalized Model and Assumptions

Suppose the system manages J heterogeneous resources, indexed by $j \in \{1, \ldots, J\}$. The reward rate for serving a request is now pair-dependent. Specifically, if request i (with intrinsic height h_i) is assigned to resource j, it generates value at a rate of $h_{i,j} = h_i \cdot q_j$, where q_j represents the resource-specific quality. This decomposable (or multiplicative) reward structure has been studied in online allocation and scheduling literature (see, e.g., Charikar et al. (2014)). Without loss of generality, we assume the resources are ordered such that $q_1 \geq q_2 \geq \cdots \geq q_J \geq 0$.

Assumption EC.1 **(Downgrade-Only)**. *Both the online algorithm and the optimal offline benchmark (*OPT*) can only downgrade the resource a request is connected to over time. That is, for any active request i, the index j of the resource serving it must be weakly increasing over time until the request completes or becomes fully unmatched.*

We emphasize that downgrade-only is a restriction coming from the operational environment: it constrains the online algorithm and the optimal offline benchmark identically, so our guarantee is measured against the strongest offline policy that also obeys it. It is, moreover, the direct generalization of the non-resumable preemption of our core model (see the special case below), and it reflects settings in which a request, once moved to a lower-quality resource, is not later upgraded, for example, when re-provisioning or resumption is impractical.

Original Model as a Special Case. Our original model, featuring a hard capacity of C homogeneous resources, is a special case of this generalized framework. By setting $q_j = 1$ for all $j \in \{1, \dots, C\}$, and $q_j = 0$ for all $j > C$, the downgrade-only assumption strictly reduces to the non-resumable preemption assumption. Specifically, once a request's resource index j is downgraded past C, its reward rate permanently drops to 0, which is mathematically equivalent to an irreversible preemption and disconnection from the capacity pool.

Building on this structure, we define $U_\pi(\mathcal{I})$ (and similarly $\mathsf{OPT}(\mathcal{I})$) to be the actual utility achieved under instance $\mathcal{I}$ by policy π. More formally, for any request i, let $j_i^\pi(t)$ denote the index of the resource serving request i at time t under policy π. By Assumption 1, $j_i^\pi(t)$ is weakly increasing over time, implying that the quality tier $q_{j_i^\pi(t)}$ experienced by request i is weakly decreasing. Thus, we formulate the total actual utility as:

$$U_\pi(\mathcal{I}) \triangleq \sum_{i \in [N]} \left(\int_{t_i}^{d_i} q_{j_i^\pi(t)} h_i \, dt \right),$$

where d_i is the time at which request i becomes unmatched under policy π.

B.2. Generalized Index-Based Algorithm

To adapt our index-based policies to this heterogeneous setting, we introduce a greedy cascading assignment mechanism. Let $f_i(t)$ denote the index value of request i at time t as defined in Subsection 3.2. If a resource j is unmatched at time t, we conventionally treat it as serving an active request with an index value of 0.

Algorithm 3 formally characterizes our generalized index-based policy, which triggers a cascading adjustment of assignments whenever a new request is admitted.

Algorithm 3 Generalized Index-Based Policy: Cascading Downgrade Assignment

At any time t when a new request i arrives:

1. **Initial Assignment:** Let S_i be the set of all resources currently serving a request with an index value strictly less than $f_i(t)$. If $S_i = \emptyset$, request i is rejected (left unmatched). Otherwise, allocate request i to the *best* (lowest indexed) server $j_1 \in S_i$.

2. **Cascading Downgrades:** Let i_1 be the request that was displaced from j_1. To downgrade i_1, define S_1 as the set of resources *worse* than j_1 (i.e., subscript $> j_1$) currently serving a request with an index strictly less than $f_{i_1}(t)$.

3. If $S_1 \neq \emptyset$, downgrade i_1 to the best server $j_2 \in S_1$. This displaces a new request i_2.

4. **Iteration:** Repeat this logic for i_2, finding the best available worse server $j_3 \in S_2$, and so forth.

This cascading process continues iteratively until some displaced request i_k finds its corresponding set S_k to be empty. At this point, i_k is left unmatched, and the assignment adjustment process at time t concludes.

B.3. Theoretical Analysis

To establish the competitive ratio of Algorithm 3, we leverage the parallel partitioning framework introduced in Section 4. Specifically, consider a parallel partitioning $\mathcal{B}$ of the plane, with r_{z_i} denoting the unique base curve that passes through the point (t_i, h_i), where t_i is the arrival time of request i and h_i is its intrinsic value. Because we are currently focusing on the case where the intrinsic value h_i remains constant over time, we impose the mild restriction that our base curves must be weakly monotonically decreasing over time.

Let γ_a and γ_o be defined exactly as in Section 4, based *solely* on the intrinsic reward of the requests (i.e., factoring out the multiplicative quality q_j of the actual utility):

$$\gamma_o \triangleq \inf_{z=(t,h)\in\mathbb{R}^2_+,\ t'\in(t,t+1)} \left\{ \frac{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''}{h(t'-t)} \right\} = \inf_{z=(t,h)\in\mathbb{R}^2_+} \left\{ \frac{\int_t^{t+1} \mathcal{B}_{r_z}(t'')\ dt''}{h} \right\},$$

$$\begin{aligned}\gamma_a &\triangleq \inf_{z=(t,h)\in\mathbb{R}^2_+} \left\{ \inf_{t'\in(t,t+1)} \left\{ \frac{h(t'-t)}{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''} \right\},\ \frac{h}{\int_t^{t+2} \mathcal{B}_{r_z}(t'')\ dt''} \right\} \\ &= \inf_{z=(t,h)\in\mathbb{R}^2_+} \left\{ 1,\ \frac{h}{\int_t^{t+2} \mathcal{B}_{r_z}(t'')\ dt''} \right\},\end{aligned}$$

where both equalities naturally follow because $\mathcal{B}_{r_z}(t'')$ is weakly monotonically decreasing with respect to t''; thus, the infimum in the top expression is attained as $t' \to t+1$, while in the bottom expression, it occurs as $t' \to t$.

Let $\pi^{\mathcal{B}}$ denote the generalized index-based policy (Algorithm 3) equipped with the index function $f_i(t) = \mathcal{B}_{r_{z_i}}(t)$ for all requests i at any time $t \in \mathbb{R}$. Analogous to Theorem 1, our objective is to prove that $U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a\gamma_o \mathsf{OPT}(\mathcal{I})$ for all $\mathcal{I}$. To accomplish this, we must first adapt the concepts of quasi-utility and the extended policy to our generalized heterogeneous setting.

First, akin to Definition 6, the quasi-utility of any policy π under any instance $\mathcal{I}$ is defined as:

$$V_{\pi,\mathcal{B}}(\mathcal{I}) \triangleq \sum_{i\in[N]} \left(\int_{t_i}^{d_i} q_{j_i^\pi(t)} \mathcal{B}_{r_{z_i}}(t)\ dt \right).$$

Second, the extended policy $\tilde{\pi}$ is defined analogously to Definition 5, with one key distinction: we extend any completed request (i.e., conceptually continue serving it for one extra unit of time) on the *final* resource it occupied prior to completion. Note that this final resource inherently possesses the lowest quality among all resources that the request was ever assigned to.

Paralleling Subsection 4.2, the proof of Theorem 1 in this generalized setting relies on three key lemmas.

LEMMA EC.1. *For any instance $\mathcal{I}$, any downgrade-only policy π, and (weakly decreasing) base $\mathcal{B}$, we have*

$$U_\pi(\mathcal{I}) \geq \gamma_a V_{\tilde{\pi},\mathcal{B}}(\mathcal{I}).$$

Proof. The proof closely mirrors its homogeneous counterpart in Subsection D.1.1. In particular, it is sufficient to show that for all $i \in [N]$, we have

$$\int_{t_i}^{d_i} q_{j_i^\pi(t)} h_i\ dt \geq \gamma_a \left(\int_{t_i}^{\tilde{d}_i} q_{j_i^\pi(t)} \mathcal{B}_{r_{z_i}}(t)\ dt \right).$$

We partition the analysis into two cases:

1. $d_i < t_i + 1$: In this case, $\tilde{d}_i = d_i$, and we can write
$$\frac{\int_{t_i}^{d_i} q_{j_i^\pi(t)} h_i \ dt}{\int_{t_i}^{d_i} q_{j_i^\pi(t)} \mathcal{B}_{r_{z_i}}(t) \ dt} \geq 1 \geq \gamma_a.$$
This holds because the ratio $\frac{q_{j_i^\pi(t)} h_i}{q_{j_i^\pi(t)} \mathcal{B}_{r_{z_i}}(t)} = \frac{h_i}{\mathcal{B}_{r_{z_i}}(t)}$ is weakly increasing as a function of t, meaning the infimum is strictly bounded from below as $d_i \to t_i$.

2. $d_i = t_i + 1$: In this case, $\tilde{d}_i = d_i + 1 = t_i + 2$, yielding:
$$\begin{aligned}
\frac{\int_{t_i}^{t_i+1} q_{j_i^\pi(t)} h_i \ dt}{\int_{t_i}^{t_i+2} q_{j_i^\pi(t)} \mathcal{B}_{r_{z_i}}(t) \ dt} &= \frac{\int_{t_i}^{t_i+1} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) h_i \ dt \ + \ q_{j_i^\pi(t_i+1)} h_i}{\int_{t_i}^{t_i+2} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) \mathcal{B}_{r_{z_i}}(t) \ dt \ + \ q_{j_i^\pi(t_i+1)} \int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t) \ dt} \\
&= \frac{\int_{t_i}^{t_i+1} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) h_i \ dt \ + \ q_{j_i^\pi(t_i+1)} h_i}{\int_{t_i}^{t_i+1} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) \mathcal{B}_{r_{z_i}}(t) \ dt \ + \ q_{j_i^\pi(t_i+1)} \int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t) \ dt} \\
&\geq \min \left\{ \frac{\int_{t_i}^{t_i+1} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) h_i \ dt}{\int_{t_i}^{t_i+1} (q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)}) \mathcal{B}_{r_{z_i}}(t) \ dt}, \frac{q_{j_i^\pi(t_i+1)} h_i}{q_{j_i^\pi(t_i+1)} \int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t) \ dt} \right\} \\
&\geq \min \left\{ 1, \frac{h_i}{\int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t) \ dt} \right\} \geq \gamma_a.
\end{aligned}$$
Here, the second equality utilizes the fact that the request is conceptually extended on its final resource for the entire duration from $t_i + 1$ to $t_i + 2$. The third inequality leverages the algebraic property that the ratio of sums is bounded by the minimum and maximum of the corresponding individual ratios. The final inequality holds because all coefficients of $(q_{j_i^\pi(t)} - q_{j_i^\pi(t_i+1)})$ are non-negative due to downgrade-only assumption, and the ratio $\frac{h_i}{\mathcal{B}_{r_{z_i}}(t)}$ is weakly increasing as a function of t.

This concludes the proof. □

LEMMA EC.2. *For any instance $\mathcal{I}$ and base $\mathcal{B}$, we have*
$$V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}) \geq V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}).$$

Proof. The proof of this lemma expands upon its homogeneous counterpart (Lemma 3), albeit with additional structural intricacies. For notational simplicity in this proof, we refer to $\widetilde{\pi^{\mathcal{B}}}$ simply as $\tilde{\pi}$. We evaluate the instantaneous quasi-utility rate of both algorithms at an arbitrary time t. Specifically, we show that the summation of the base curve values (weighted by their matched resource qualities) for the requests served by $\tilde{\pi}$ is at least that of the requests served by OPT at any given time t:
$$\sum_{i \in \mathcal{H}_{\tilde{\pi}}(t)} q_{j_i^{\tilde{\pi}}(t)} \mathcal{B}_{r_i}(t) \geq \sum_{i \in \mathcal{H}_{\mathsf{OPT}}(t)} q_{j_i^{\mathsf{OPT}}(t)} \mathcal{B}_{r_i}(t). \tag{EC.1}$$

Because OPT can serve at most one request per resource, let $i_j^{\mathsf{OPT}}(t) \in \mathcal{H}_{\mathsf{OPT}}(t)$ denote the unique request that OPT is serving on resource j at time t. Similarly, let $i_j^{\tilde{\pi}}(t) \triangleq \{i \in \mathcal{H}_{\tilde{\pi}}(t) | j_i^{\tilde{\pi}}(t) = j\}$ denote the set of all requests that policy $\tilde{\pi}$ is associating with resource j at time t. Note that $i_j^{\tilde{\pi}}(t)$ may encompass a single actual active request, alongside potentially multiple extended requests that completed at resource j during the interval $(t-1, t]$. Using this notation, we can rewrite Equation (EC.1) as:
$$\sum_{j \in [J]} \left(q_j \sum_{i \in i_j^{\tilde{\pi}}(t)} \mathcal{B}_{r_i}(t) \right) \geq \sum_{j \in [J]} q_j \mathcal{B}_{r_{i_j^{\mathsf{OPT}}(t)}}(t). \tag{EC.2}$$

Suppose we sort the base indices $\{r_i\}_{i \in \mathcal{H}_{\mathsf{OPT}}(t)}$ in descending order, denoted by the sequence $(r_{\sigma_1}, r_{\sigma_2}, \ldots, r_{\sigma_J})$. We assert the following claim:

CLAIM EC.1. *At any time t and $\forall j \in [J]$, we have*

$$\max_{i \in i_j^{\tilde{\pi}}(t)} r_i \geq r_{\sigma_j},$$

which immediately implies that $\max_{i \in i_j^{\tilde{\pi}}(t)} \mathcal{B}_{r_i}(t) \geq \mathcal{B}_{r_{\sigma_j}}(t)$.

Assuming this claim holds, we can readily establish Equation (EC.2) through the following chain of inequalities:

$$\sum_{j \in [J]} \left(q_j \sum_{i \in i_j^{\tilde{\pi}}(t)} \mathcal{B}_{r_i}(t) \right) \geq \sum_{j \in [J]} \left(q_j \max_{i \in i_j^{\tilde{\pi}}(t)} \mathcal{B}_{r_i}(t) \right) \geq \sum_{j \in [J]} q_j \mathcal{B}_{r_{\sigma_j}}(t) \geq \sum_{j \in [J]} q_j \mathcal{B}_{r_{i_j^{\mathsf{OPT}}(t)}}(t).$$

Here, the first inequality relies on the non-negativity of the base curves; the second directly applies the claim; and the final inequality leverages the established rearrangement inequality—sorting two non-negative sequences in descending order uniquely maximizes their pair-wise dot product across all possible permutations.

We now prove the central claim.

Proof of Claim EC.1. We proceed by contradiction. Suppose there exists a resource $j \in [J]$ at time t such that:

$$\max_{i \in i_j^{\tilde{\pi}}(t)} r_i < r_{\sigma_j}.$$

Observe two structural properties maintained by our generalized index-based policy (Algorithm 3): (1) *Weak Monotonicity*: the index of the base curve assigned to any resource j is weakly increasing over time (prior to completion), because replacements exclusively occur via higher-indexed requests; and (2) *Greedy Cascading*: any displaced request is systematically pushed to the best available lower-tier resource among those serving a weakly lower base value.

By the Weak Monotonicity property, if at any time $t' \in (t-1, t]$ our policy is serving a request i with base index r_i on resource j, then either this request is successively replaced by requests with weakly higher base indices, or it completes at some time $t'' \in [t', t]$. In the event of completion, the extended policy $\tilde{\pi}$ sustains its value for an additional unit of time, extending beyond t. Formally, this yields:

$$\max_{t' \in (t-1,t]} r_{i_j^{\pi_{\mathcal{B}}}(t')} = \max_{i \in i_j^{\tilde{\pi}}(t)} r_i < r_{\sigma_j}.$$

Given this inequality, and knowing that the j requests corresponding to $(r_{\sigma_1} \geq \cdots \geq r_{\sigma_j})$ must have arrived within the interval $(t-1, t]$, it follows that these j requests have possessed strictly higher base indices than the occupying requests at resource j for their entire duration up to time t. Consequently, all j of these requests must have been assigned to resources of a strictly higher tier than j upon arrival. Even if they experienced subsequent downgrades, they must have remained strictly above resource j up to time t; had they ever dropped out of top $j-1$ resources, the Greedy Cascading property would dictate they should first displace the lower-valued request at resource j, as j is always their preferred resource compared to anything other than top $j-1$ resources.

This creates an unavoidable contradiction: there are only $j-1$ resources better than j, yet they must simultaneously accommodate all j requests corresponding to $(r_{\sigma_1}, \ldots, r_{\sigma_j})$. This violates the hard constraint that each resource serves at most one request at a time, completing the proof of the claim. □

LEMMA EC.3. *For any instance $\mathcal{I}$ and (weakly decreasing) base $\mathcal{B}$, we have*

$$V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) \geq \gamma_o \mathsf{OPT}(\mathcal{I}).$$

Proof. The logic closely tracks the homogeneous version in Subsection D.1.3. Specifically, it suffices to prove that for all $i \in [N]$, we have:

$$\int_{t_i}^{d_i^*} q_{j_i^{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t)\ dt \geq \gamma_o \left(\int_{t_i}^{d_i^*} q_{j_i^{\mathsf{OPT}}(t)} h_i\ dt \right).$$

Crucially, we note two facts: (i) the ratio $\frac{\mathcal{B}_{r_{z_i}}(t)}{h_i}$ is a weakly decreasing function of t, and (ii) the quality multiplier $q_{j_i^{\mathsf{OPT}}(t)}$ is a weakly decreasing function of t due to the downgrade-only assumption. Therefore, we can write:

$$\frac{\int_{t_i}^{d_i^*} q_{j_i^{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t)\ dt}{\int_{t_i}^{d_i^*} q_{j_i^{\mathsf{OPT}}(t)} h_i\ dt} \geq \frac{\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt}{\int_{t_i}^{d_i^*} h_i\ dt} \geq \frac{\int_{t_i}^{t_i+1} \mathcal{B}_{r_{z_i}}(t)\ dt}{\int_{t_i}^{t_i+1} h_i\ dt} \geq \gamma_o,$$

which validates the bound and completes the proof. □

By sequentially chaining the inequalities established in Lemmas EC.1, EC.2, and EC.3, we establish the generalized counterpart to Theorem 1.

THEOREM EC.1. *Under the generalized heterogeneous resource model, for any (weakly decreasing) base $\mathcal{B}$, the generalized index-based policy $\pi^{\mathcal{B}}$ (Algorithm 3) achieves a competitive ratio of at least $\gamma_a\gamma_o$. That is,*

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a \gamma_o \mathsf{OPT}(\mathcal{I}) \quad \forall \mathcal{I}.$$

Proof. Combining Lemmas EC.1, EC.2, and EC.3 yields the required sequence of bounds:

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}) \geq \gamma_a V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) \geq \gamma_a\gamma_o \mathsf{OPT}(\mathcal{I}). \quad \square$$

This directly guarantees the competitive ratio $\gamma_a\gamma_o$, seamlessly extending our analytical framework to environments with heterogeneous resources and soft capacity. Consequently, we can directly import the exponential base curves from Subsection 4.3 to achieve the identical competitive ratios established in the homogeneous setting. Specifically, let $\pi^{\exp(\alpha^*)}$ denote the generalized index-based policy (Algorithm 3) operating with the exponential index function:

$$f_i(t) \triangleq h_i e^{-\alpha^*(t-t_i)},$$

where α^* is defined as in Equation (10). We then obtain the following performance guarantee:

COROLLARY EC.1. *The competitive ratio $\mathcal{CR}$ of the generalized index-based policy $\pi^{\exp(\alpha^*)}$ satisfies:*

$$\mathcal{CR}(\pi^{\exp(\alpha^*)}) = \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68.$$

REMARK EC.1 (ROBUSTNESS TO DYNAMIC RESOURCE PROFILES). As a direct generalization of Remark 2 from the homogeneous setting, our competitive ratio guarantees for the heterogeneous model remain rigorously robust to dynamic capacity environments. While our exposition assumes a static hierarchy of J resources, our generalized analysis relies exclusively on instantaneous, pointwise bounds on the quasi-utility rate (e.g., Lemma EC.2). Consequently, our results hold even if the availability (capacity) or the quality q_j of the various resource tiers fluctuates arbitrarily over time, provided the algorithm dynamically adjusts assignments according to the current capacity profile while strictly adhering to the downgrade-only restriction (i.e., active requests can only be shifted to weakly lower-tier resources or permanently discharged).

REMARK EC.2 (COMPATIBILITY WITH TIME-VARYING REWARDS). The proof framework developed in this section focuses on constant intrinsic heights h_i strictly for simplicity of exposition. By defining the appropriate exponential base curves analogously to Section 7, this heterogeneous resource extension can be generalized to the time-varying reward rates setting of Section 7.

This extension illustrates that our framework and guarantees are robust to a particular heterogeneous-resource (soft-capacity) structure, in which a request displaced from a higher-quality tier may continue on a lower one. It is distinct from adaptively choosing overflow capacity at an explicit cost or penalty, which we do not claim to resolve and regard as a separate, interesting direction for future work.

Appendix C: Proofs of Section 3

C.1. Proofs in Subsection 3.3

C.1.1. Proof of Lemma 1

Proof of Lemma 1. To prove this, we first provide an adjustment to the algorithm π^{h}, namely $\widetilde{\pi^{\mathsf{h}}}$. The departure decisions in $\widetilde{\pi^{\mathsf{h}}}$ are exactly same as in π^{h}, except for requests that π^{h} keeps until their deadline, i.e., for a whole duration of 1. For such requests, we assume that $\widetilde{\pi^{\mathsf{h}}}$ keeps them twice as much, i.e., for a total duration of 2. Notice that neither the capacity constraint nor the deadline constraint needs to be satisfied for $\widetilde{\pi^{\mathsf{h}}}$, as this policy is only for the sake of analysis and will not be used for implementation. Lastly, we will prove the following two inequalities:

1. $2\, U_{\pi^{\mathsf{h}}}(\mathcal{I}) \geq U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I})$:

The proof of this inequality simply comes from the definition of $U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I})$, as all requests that are preempted before their deadline under π^{h} will be served for the same duration under $\widetilde{\pi^{\mathsf{h}}}$ as well, and all those which are kept until their deadline, i.e., for duration 1, will now be kept for duration 2 under $\widetilde{\pi^{\mathsf{h}}}$. Therefore, for each request, $\widetilde{\pi^{\mathsf{h}}}$ will get at most twice accumulated reward as π^{h}, now if we sum over all requests we will get the inequality we wanted.

2. $U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I}) \geq \mathsf{Skyline}(\mathcal{I})$:

We claim that at any time t, $\widetilde{\pi^{\mathsf{h}}}$ is serving requests with rewards that dominates that of $\mathsf{Skyline}(\mathcal{I})$. More specifically, at any time t, let $h_{s_1} \geq h_{s_2} \geq ... \geq h_{s_k}, (k \leq C)$, be the distinct requests that $\mathsf{Skyline}(\mathcal{I})$ is keeping. Then we claim that there are distinct requests $h_{i_1} \geq h_{i_2} \geq ... \geq h_{i_k}$ that $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ is serving which satisfies $\forall 1 \leq j \leq k,\ h_{i_j} \geq h_{s_j}$.

First notice that if $\pi^{\mathsf{h}}(\mathcal{I})$ is serving some request at time t, $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ does so as well. Now among the sequence $h_{s_1} \geq h_{s_2} \geq ... \geq h_{s_k}$ consider the request with highest reward that $\pi^{\mathsf{h}}(\mathcal{I})$ is not serving at time t, say s_j. This means that $\pi^{\mathsf{h}}(\mathcal{I})$ must have preempted request s_j at some time $t' \leq t$. This implies that, at the time t', $\pi^{\mathsf{h}}(\mathcal{I})$ was keeping C requests all with rewards at least h_{s_j}. Now, look at the time interval $[t', t]$. First, we know that, unless hitting the deadline, $\pi^{\mathsf{h}}(\mathcal{I})$ will only replace a request with another one with higher reward rate. Second, if a request hit its deadline while $\pi^{\mathsf{h}}(\mathcal{I})$ was keeping it, then $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ will keep it for a whole extra unit of time. Therefore, any of such higher reward requests that $\pi^{\mathsf{h}}(\mathcal{I})$ is keeping at time t' will either remain to be served until t, replace with some higher reward (could be multiple replaces over the time interval until t, but each of them has higher reward rate with the previous one in this "replacement chain"), or hit the deadline. Notice that in all of these three cases, as $t - t' < 1$, $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ will serve at least C number of

requests at time t which weakly dominates (in terms of their reward rate) those ones that $\pi^{\mathsf{h}}(\mathcal{I})$ was serving at time t', which themselves weakly dominated the s_j that $\mathsf{Skyline}(\mathcal{I})$ is serving at time t.
Finally, for $s_1, ..., s_{j-1}$ we knew that $\pi^{\mathsf{h}}(\mathcal{I})$ and so as $\widetilde{\pi}^{\mathsf{h}}(\mathcal{I})$ are serving them until at least time t (by definition of s_j). Combining this with the statement that we showed earlier - i.e., $\widetilde{\pi}^{\mathsf{h}}(\mathcal{I})$ is keeping at least C requests with rewards at least h_{s_j} (which is greater or equal than the reward rate of the rest of the requests s_{j+1} to s_k) - we can conclude the proof of this dominance claim (and so as the second inequality) by setting $i_l = s_l,\ \forall 1 \le l \le j-1$, and then choosing $k-(j-1)$ highest reward requests from the rest of the requests that $\widetilde{\pi}^{\mathsf{h}}$ is serving at time t - each of which had a reward rate of at least h_{s_j}.

□

C.1.2. Proof of Proposition 1

Proof of Proposition 1. The proof of $\mathcal{CR}(\pi^{\mathsf{h}}) \ge \frac{1}{2}$ is readily attained from Lemma 1. The rest is to show that this ratio is tight, by providing a simple bad instance. Instance $\mathcal{I}$ consists of only 2 types of requests, where each type has C identical requests. The first type is $z_1 := (t_1 = 0, h_1 = 1)$, and the second type is $z_2 := (t_2 = 1^-, h_2 = 1^-)$. Under this instance $\mathcal{I}$, π^{h} will only serve all the first types till their deadline ($t = 1$) and preempts all second types immediately, whereas the optimal offline $\mathsf{OPT}(\mathcal{I})$ will preempt all the first types when second types arrive and serve them till their deadline ($t = 2^-$). It is clear to see that $U^{\mathsf{h}}_{\pi}(\mathcal{I}) = C$ and $\mathsf{OPT}(\mathcal{I}) = C^- + C^-$, which results in:

$$\mathcal{CR}(\pi^{\mathsf{h}}) \le \frac{U^{\mathsf{h}}_{\pi}(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})} = \frac{1}{2},$$

concluding the tightness of the analysis, i.e., $\mathcal{CR}(\pi^{\mathsf{h}}) = \frac{1}{2}$. □

C.2. Proofs in Subsection 3.4

We will prove Proposition 2 in this subsection.

First, we will provide the definitions and notations that we use to analyse the competitive ratio of the policy π^{ra}. It is true that the algorithm has no non-trivial information about how the optimal offline policy is making its decisions; nonetheless, we can use the decisions of the optimal offline in order to provide fitting structure for our competitive ratio analysis of π^{ra} against the unknown-to-algorithm optimal offline.

In the following we will construct a continuum of bijective functions, $\forall t \in \mathbb{R}$, $\mu_t : \mathcal{H}_{\mathsf{OPT}}(t) \to \mathcal{H}_{\pi^{\mathsf{ra}}}(t)$, which are one-to-one correspondences that match each request that is being served by OPT at time t, with a distinct request that is being served by π^{ra} at the same time t. The main purpose of this one-to-one correspondence is that it provides us with a clear way of accounting the rewards for the requests that π^{ra} is serving and then comparing them with the rewards for the requests that OPT is serving them, without being prone to double counting mistakes. We also denote the inverse of this bijective matching at each time t by $\mu_t^- : \mathcal{H}_{\pi^{\mathsf{ra}}}(t) \to \mathcal{H}_{\mathsf{OPT}}(t)$.

Now let us show how we will construct this one-to-one correspondence at each time. Notice that whenever a new request arrives, as long as there is some capacity available, both of OPT and π^{ra} will start serving that. Additionally, both of these policies face the same instance, meaning that both of them will see the same arrival time and lifespan for each request. This will tell us that since time $t = 0$, as long as the capacity constraint is not binding, both of them will serve the exact same set of requests at each time. Let

us denote t_0 as the first time that the capacity constraint will become binding. With this in mind, we set $\mu_t(i) = i \in \mathcal{H}_{\pi^{\mathsf{ra}}}(t), \forall t \in [0, t_0), i \in \mathcal{H}_{\mathsf{OPT}}(t)$.

In the time intervals that there is no change to either of $\mathcal{H}_{\pi^{\mathsf{ra}}}(t)$ and $\mathcal{H}_{\mathsf{OPT}}(t)$, we also do not change the matching $\mu_t(.)$. Also, notice that whenever a request hits its deadline either of the two policies has to remove it, but we can think of it as they are replacing it with a request with zero reward. Moreover, the index (remaining area) of such a request at the time of its deadline is also zero. Therefore, as for the π^{ra}, we can even view this removal as a deliberate action to a request with same zero index (i.e., to a zero reward request) rather than viewing it such that the algorithm is forced to remove that request whose lifespan is finished. With this in mind, the remaining part for us to show in order to fully characterize $\mu_t(.)$ at all times is to show how $\mu_t(.)$ will change at times when either π^{ra} or OPT are accepting a new arrival.

Suppose request i arrives to the system. There are 4 possible cases that can happen, and we will explicitly state how the matching should adapt in each of them.

1. Neither OPT nor π^{ra} accepts request i: In this case, μ_{t_i} continues to be what it was before t_i without any change.

2. Only OPT accepts i: In this case, consider the request which would be replaced with i by OPT at time t_i. Let us call that request j. The matching at this time, μ_{t_i}, will change in the following manner: we connect i to what j was matched to, i.e., $\mu_{t_i}(i) \leftarrow \mu_{t_i^-}(j)$, and we do not change anything else.

3. Only π^{ra} accepts i: In this case, we do a similar change as in the previous case, but now on the π^{ra} side. More specifically, let j to be the request that is being replaced with i by π^{ra} at time t_i. Also let k to be the match of j right before time t_i, i.e., $k = \mu^-_{t_i^-}(j)$. Then, we will change the match of k at t_i from j to be i, $\mu_{t_i}(k) \leftarrow i$. Again, everything else will remain unchanged.

4. Both π^{ra} and OPT accepts i: In this case, let j_π and j_{OPT} be the requests that are being replaced with i at time t_i by π^{ra} and OPT, respectively. There are two possible scenarios that might have happened:

   (a) $\mu_{t_i^-}(j_{\mathsf{OPT}}) = j_\pi$: In this scenario, both of these requests will be discarded by the corresponding policies, and both policies will accept i. As a result the only change we need to do is to set $\mu_{t_i}(i) = i$.

   (b) $\mu_{t_i^-}(j_{\mathsf{OPT}}) \neq j_\pi$: In this scenario, not only do we need to set $\mu_{t_i}(i) = i$, but also we have to match together the matches of both j_{OPT} and j_π, i.e., $\mu_{t_i}(k) \leftarrow \mu_{t_i^-}(j_{\mathsf{OPT}})$ where $k = \mu^-_{t_i^-}(j_\pi)$.

Notice that we also discard the requests from domain of the matching for requests that get preempted since their departure time.

REMARK EC.3. Note that from the way we construct matching μ_t, we can easily deduce that it would be, indeed, a one-to-one correspondence from $\mathcal{H}_{\mathsf{OPT}}(t)$ to $\mathcal{H}^{\mathsf{ra}}_{\pi}(t)$.

Now that we have built our matching μ_t over time, let us also define another function that determines for any request i what is the sequence of its matched requests (on the π^{ra}'s side) over the time interval it was being served by OPT. More formally, for any request i we define $\sigma_i : [t_i, d_i^*) \to [N]$, where d_i^* was the departure time of i by OPT. For any request i, we will set:

$$\sigma_i(t) \triangleq \mu_t(i), \quad \forall t \in [t_i, d_i^*).$$

With this way of matching the requests that OPT is keeping with the ones that π^{ra} is keeping, we will have the following properties hold:

CLAIM EC.2. *For any request $i \in [N]$ such that $t_i < d_i^*$, the following two properties hold for the requests in the sequence $\sigma_i(.)$:*

1. *At time t_i, the remaining area of the request $\sigma_i(t_i)$ is at least as large as that of request i:*

$$h_{\sigma_i(t_i)}(t_{\sigma_i(t_i)} + 1 - t_i) \geq h_i.$$

2. *At any time $t \in (t_i, d_i^*)$, the remaining area of the request $\sigma_i(t)$ at the time t is at least as large as that of the request $\sigma_i(t^-)$:*

$$h_{\sigma_i(t)}(t_{\sigma_i(t)} + 1 - t) \geq h_{\sigma_i(t^-)}(t_{\sigma_i(t^-)} + 1 - t).$$

Proof of Claim EC.2. The proof follows from the way we have constructed the matching, as well as the fact that the algorithm always keeps the requests with the highest remaining area. Now let us formally state the proof for each part of the claim:

1. Request i arrives at time t_i, now there are two cases:

(a) if algorithm also accepts to serve i (case 4.a): in this case by definition we have $\sigma_i(t_i) = i$ and the statement of this part holds clearly.

(b) if algorithm never serve request i (immediate preempt of i by algorithm, case 2): in this case, we did $\sigma_i(t_i) = \mu_{t_i}(i) \leftarrow \mu_{t_i^-}(j)$. From the fact that request $\mu_{t_i^-}(j)$ is still kept in the system and request i was preempted by π^{ra}, we get that its remaining area at time t_i should have been higher, which is exactly what we wanted to show.

2. To show this part, if $\sigma_i(t^-) = \sigma_i(t)$, then clearly the expression holds with equality. On the other hand, if $\sigma_i(t^-) \neq \sigma_i(t)$, it means that either case 3 or case 4.b) has happened, and in both of them π^{ra} preempts $\sigma_i(t^-)$ instead of $\sigma_i(t)$. Thus, again by the fact that π^{ra} keeps the requests with higher remaining area at t, we immediately get: $h_{\sigma_i(t)}(t_{\sigma_i(t)} + 1 - t) \geq h_{\sigma_i(t^-)}(t_{\sigma_i(t^-)} + 1 - t)$.

□

The next lemma will provide a comparison between the total reward that OPT is acquiring from any request i and the reward that π^{ra} is accumulating over that period of time from the requests that are matched to i, namely $\sigma_i(.)$.

LEMMA EC.4. *For any request $i \in [N]$ such that $t_i < d_i^*$, the following inequality holds:*

$$\int_{t_i}^{d_i^*} h_{\sigma_i(t)} \; dt \geq (1 - \frac{1}{e}) \times h_i(d_i^* - t_i).$$

Proof of Lemma EC.4. To prove this lemma, we make use of Claim EC.2, but before that let us define some simplifying notation. In the sequence of requests $\sigma_i(.)$, let us define τ_j as the j-th changing time in the sequence, with $\tau_0 \triangleq t_i$ and $\tau_{n_i} \triangleq d_i^*$, where n_i is the total number of matches to request i. In other words, τ_j is the j-th time in the sequence that $\sigma_i(\tau_j) \neq \sigma_i(\tau_j^-)$. Let us also define $\forall 1 \leq j \leq n_i : \alpha_j \triangleq \tau_j - \tau_{j-1}, \beta_j \triangleq (1 + t_{\sigma_i(\tau_{j-1})} - \tau_{j-1})$, where α_j would be the length of time that i is matched to its j-th match and β_j would be the remaining lifespan of i's j-th match at the time when its matched interval starts (at time τ_{j-1}). Additionally, for conciseness we refer to $h_{\sigma_i(\tau_{j-1})}$ as h_{i_j}. With these in mind, from Claim EC.2 we have:

$$h_{i_1}\beta_1 \geq h_i,$$
$$h_{i_{j+1}}\beta_{j+1} \geq h_{i_j}(\beta_j - \alpha_j) = h_{i_j}\beta_j(1 - \frac{\alpha_j}{\beta_j}) \quad \forall 1 \leq j \leq n_i - 1,$$

where the first inequality is equivalent to the first part of the claim, and the second inequality will be derived by setting $t=\tau_j$ for all $1\le j\le n_i-1$, in the second part of the claim. By using the second inequality in a cascade manner, we can get:

$$\forall 1\le j\le n_i-1:\ h_{i_{j+1}}\beta_{j+1}\ge h_{i_1}\beta_1\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\ \Rightarrow\ h_{i_{j+1}}\alpha_{j+1}\ge h_{i_1}\beta_1\left(\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\right)\frac{\alpha_{j+1}}{\beta_{j+1}}.$$

Then, if we sum up the above inequality for all j from 0 to n_i-1 we get:

$$\begin{aligned}\int_{t_i}^{d_i^*}h_{\sigma_i(t)}\ dt&=\sum_{j=0}^{n_i-1}h_{i_{j+1}}\alpha_{j+1}\ge h_{i_1}\beta_1\sum_{j=0}^{n_i-1}\left(\left(\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\right)\frac{\alpha_{j+1}}{\beta_{j+1}}\right)\\&=h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\frac{\alpha_j}{\beta_j})\right)\ge h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\alpha_j)\right),\end{aligned}$$

where the second inequality uses the fact that $\alpha_j\le\beta_j\le 1$, which is true by definition as $t_{\sigma_i(\tau_{j-1})}\le\tau_{j-1}\le\tau_j\le t_{\sigma_i(\tau_{j-1})}+1$.

Moreover, notice that we have $\sum_{j=1}^{n_i}\alpha_j=d_i^*-t_i$, so we can further lower bound the last inequality by considering the maximum possible value for the product expression, which will happen when all of its terms are equal to each other, i.e., $\forall j:\alpha_j=\frac{d_i^*-t_i}{n_i}$. Lastly, by using inequalities $h_{i_1}\beta_1\ge h_i$ and $\forall x: e^x\ge 1+x$, we will get:

$$\int_{t_i}^{d_i^*}h_{\sigma_i(t)}\ dt\ge h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\alpha_j)\right)\ge h_{i_1}\beta_1\left(1-(1-\frac{d_i^*-t_i}{n_i})^{n_i}\right)\ge h_i\left(1-e^{-(d_i^*-t_i)}\right).$$

Finally, the function $\frac{1-e^{-x}}{x}$ is a decreasing function, thereby finishing the proof (as $d_i^*-t_i\le 1$):

$$\int_{t_i}^{d_i^*}h_{\sigma_i(t)}\ dt\ge h_i\left(1-e^{-(d_i^*-t_i)}\right)\ge h_i(d_i^*-t_i)(1-\frac{1}{e}).$$

□

We are now ready to finish the proof of Proposition 2, in which we prove the lower bound of $1-\frac{1}{e}$ as well as provide the tight upper bound (bad instance) for it.

Proof of Proposition 2. Due to the fact that μ_t was a one-to-one correspondence at all time, we are sure that at any point in time t, we do not double count any request that π^{ra} is keeping for different i in $\sigma_i(t)$. As a result, we can write the following:

$$\forall\mathcal{I}:\quad U_\pi^{\mathsf{ra}}(\mathcal{I})=\sum_{\{i\in[N]|t_i<d_i^*\}}\left(\int_{t_i}^{d_i^*}h_{\sigma_i(t)}\ dt\right).$$

Remember that $\mathsf{OPT}(\mathcal{I})=\sum_{i\in[N]}h_i(d_i^*-t_i)=\sum_{\{i\in[N]|t_i<d_i^*\}}h_i(d_i^*-t_i)$, so in order to get any competitive ratio X for π^{ra}, it is sufficient to show that $\forall i\in[N]$ s.t. $t_i<d_i^*$:

$$\frac{\int_{t_i}^{d_i^*}h_{\sigma_i(t)}\ dt}{h_i(d_i^*-t_i)}\ge X.$$

From Lemma EC.4 we know that the inequality holds when $X=1-\frac{1}{e}$, thereby concluding that $\mathcal{CR}(\pi^{\mathsf{ra}})\ge 1-\frac{1}{e}$.

Finally, to see that the competitive ratio of π^{ra} is tight, i.e., $\mathcal{CR}(\pi^{\mathsf{ra}})=1-\frac{1}{e}$, we will provide the following worst-case instance. The instance $\mathcal{I}$ consists of a repeated sequence of continuum arrival of requests. More

specifically, for any round k corresponding to the time interval $[k, k+1)$, we have a continuum of arrivals in the following manner: at time $k+t \in [k, k+1), \forall t \in [0,1)$, we input a request with height e^{-t}, i.e., $z_{t+k} = (t+k, e^{-t})$. Because the gradient of the height function (e^{-t}) for any $t \in (0,1)$ is equal to negative of the height of the request z_{t+k} for any round k, which itself is equal to the gradient of the index function of request z_{t+k}, we can conclude that the policy π^{ra} will accept the request z_{t+k+dt}. As for the requests z_k, i.e., when $t=0$, we know that the algorithm will accept it because $h_{z_k} = 1$, whereas $h_{z_{k^-}} = \frac{1}{e}$ which is the request that π^{h} is serving at the end of the previous round. On the other hand, the optimal offline will only serve requests z_k, and preempts all the rest of requests z_{k+t}, $\forall k \in \mathbb{Z}_+, t \in (0,1)$ immediately as they arrive. Note that if $C > 1$, we can just duplicate each request C times, and all the above argument will still hold. This way $\mathsf{OPT}(\mathcal{I})$ is always serving C requests with reward 1 at each time t (only all the integer requests), while $\pi^{\mathsf{ra}}(\mathcal{I})$ is accepting to serve all requests for infinitesimal amount of time. Thus, the total reward in each interval $[k, k+1), \forall k$ of $\mathsf{OPT}(\mathcal{I})$ would be C, and that of $\pi^{\mathsf{ra}}(\mathcal{I})$ would be equal to $C\int_0^1 e^{-t}dt = C(1-\frac{1}{e})$, which proves the tightness of $1-\frac{1}{e}$ for the competitive ratio, i.e., $\mathcal{CR}(\pi^{\mathsf{ra}}) = 1-\frac{1}{e}$.

□

Appendix D: Proofs of Section 4

D.1. Proofs in Subsection 4.2

For simplicity, we slightly abuse the notation and refer to $\widetilde{\pi^{\mathcal{B}}}$ as just $\tilde{\pi}$, in the rest of this subsection.

D.1.1. Proof of Lemma 2

Proof of Lemma 2. First notice that we have: $U_{\pi^{\mathcal{B}}}(\mathcal{I}) = \sum_{i\in[N]} h_i(d_i - t_i)$, $V_{\tilde{\pi},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\, dt\right)$. Hence, it is sufficient to show that for all $i \in [N]$, we have

$$h_i(d_i - t_i) \geq \gamma_a \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\, dt\right). \tag{EC.3}$$

Now let us consider two cases:

1. $d_i < t_i + 1$: In this case, we have $\tilde{d}_i = d_i$. From the definition of γ_a in (6), if we set $z = z_i$ and $t' = d_i \in [t_i, t_i+1)$, then it follows immediately from the first term in *min* of γ_a:

$$\frac{h_i(d_i - t_i)}{\int_{t_i}^{d_i} \mathcal{B}_{r_{z_i}}(t)\, dt} \geq \gamma_a,$$

which implies (EC.3).

2. $d_i = t_i + 1$: In this case, we have $\tilde{d}_i = d_i + 1 = t_i + 2$. Similarly, from the definition of γ_a in (6), if we set $z = z_i$, then it follows immediately from the second term in *min* of γ_a:

$$\frac{h_i}{\int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t)\, dt} \geq \gamma_a,$$

which implies (EC.3).

The proof is completed. □

D.1.2. Proof of Lemma 3

Before going into the proof, let us first state the following remark.

Remark EC.4. We can assume w.l.o.g. that the system is always full under $\pi^{\mathcal{B}}$, as we can add at any time any arbitrary number of dummy requests whose heights are all equal to 0.

Proof of Lemma 3. We can rewrite the LHS and RHS of (8) in the following manner:

- $V_{\tilde{\pi},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right) = \int_0^{T+1} \left(\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t)\right)\ dt \geq \int_0^{T} \left(\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t)\right) dt$;
- $V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) = \int_0^{T} \left(\sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t)\right)\ dt$.

Thus, it is sufficient to show that for all $t\in[0,T)$, we have

$$\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t) \geq \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t). \tag{EC.4}$$

Claim EC.3. *Given any time t and any request z_k, if we have*

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)} \left\{\mathcal{B}_{r_{z_i}}(t)\right\} \geq \mathcal{B}_{r_{z_k}}(t), \tag{EC.5}$$

then we also have

$$\forall t'\in[t,t+1): \min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\, j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}} \left\{\mathcal{B}_{r_{z_i}}(t')\right\} \geq \mathcal{B}_{r_{z_k}}(t').$$

Proof of Claim EC.3. The following fact comes from (5):

$$\exists t \text{ such that } \mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t) \iff r_{z_i} \geq r_{z_k} \iff \mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t)\ \forall t.$$

Combining the above with the assumption in (EC.5), we get:

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)} \{r_{z_i}\} \geq r_{z_k}.$$

Now notice that from the behavior of the index-based policy $\pi^{\mathcal{B}}$ we know that, unless any of its requests hit the end of its lifespan and the policy is forced to preempt that request, $\pi^{\mathcal{B}}$ preempts a request k at some time t only if there is a newly-arrived request i whose index at the time is higher than that of request k's, i.e., $f_{z_i}(t) \geq f_{z_k}(t)$ - which itself is equivalent to $\mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t)$, or simply $r_{z_i} \geq r_{z_k}$.

Now let us prove the claim. Fix an arbitrary t and $t'\in[t,t+1)$, define $D[t,t']$ as the set of the requests that $\pi^{\mathcal{B}}$ removes in the time interval $[t,t']$ due to the reason they have hit the end of their lifespans. From the way we defined $\tilde{\pi}$ we know that it will serve all these requests for an extra unit of time after their deadline. From this we can deduce that because $t'\in[t,t+1)$, then all of $D[t,t']$ would be still being served by $\tilde{\pi}$ at time t', i.e., $D[t,t']\subseteq\mathcal{H}_{\tilde{\pi}}(t')$.

Let us first define relation $\preceq$ between two sets A and B. We say $A\preceq B$, if $|A|\leq|B|$ and if we rank the requests in each of them based on the requests' r_z, then $\forall l\in[|A|]$, the $l-th$ highest r_z in A is no greater than the $l-th$ highest r_z in B.

Now consider $\mathcal{H}_{\pi^{\mathcal{B}}}(t)$. This set consists of C number of requests. Until time t' each of those requests might experience 3 types of fate: i) remain in the system, ii) might get replace multiple times (without ever - the original request or any of its replacement chain - hitting their deadline), and iii) might hit their deadline after some (possibly 0) number of replacements (and probably get replaced by a zero-reward request). Let us name the set of requests in each of these fate classes as F_1,F_2,F_3, respectively. First, by definition of $\pi^{\mathcal{B}}$, we have $\forall s:\mathcal{H}_{\pi^{\mathcal{B}}}(s)\subseteq\mathcal{H}_{\tilde{\pi}}(s)$. Therefore, we immediately get: $F_1\subseteq\mathcal{H}_{\pi^{\mathcal{B}}}(t')\subseteq\mathcal{H}_{\tilde{\pi}}(t')$. Second, we get: $F_3\preceq D[t,t']\subseteq\mathcal{H}_{\tilde{\pi}}(t')\backslash\mathcal{H}_{\pi^{\mathcal{B}}}(t')$, because of the fact that, in each of the replacement chains, the r_z sequence is increasing, and the final element of the chain is indeed in $D[t,t']$ by definition. Lastly, as for the second type,

F_2, again each of the chains will have an increasing sequence of r_z, and for this type, the chain is still alive by time t', meaning the elements of these chains at time t' is also an element of $\mathcal{H}_{\tilde{\pi}}(t')\backslash\{\mathcal{H}_{\pi^{\mathcal{B}}}(t')\cup D[t,t']\}$. Therefore, we get $F_2 \preceq \mathcal{H}_{\tilde{\pi}}(t')\backslash\{\mathcal{H}_{\pi^{\mathcal{B}}}(t')\cup D[t,t']\}$.

Combining all these three pieces together, we can get: $\mathcal{H}_{\pi^{\mathcal{B}}}(t) \preceq \mathcal{H}_{\tilde{\pi}}(t')$, and because size of $\mathcal{H}_{\pi^{\mathcal{B}}}(t)$ was C, then together with that "equivalent" assumption of the claim we can get:

$$r_{z_k} \le \min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)}\{r_{z_i}\} \le \min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\, j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}}\{r_{z_i}\},$$

which implies

$$\min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\, j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}}\{\mathcal{B}_{r_{z_i}}(t')\} \ge \mathcal{B}_{r_{z_k}}(t').$$

The proof of the claim is completed. □

Now let us prove equation (EC.4) with the help of Claim EC.3 for any arbitrary time t.

We divide $\mathcal{H}_{\mathsf{OPT}}(t)$ into two subset, $\mathcal{J} \triangleq \mathcal{H}_{\mathsf{OPT}}(t)\cap\mathcal{H}_{\pi^{\mathcal{B}}}(t)$ and $\mathcal{D} \triangleq \mathcal{H}_{\mathsf{OPT}}(t)\setminus\mathcal{H}_{\pi^{\mathcal{B}}}(t)$. Now notice for any $k\in\mathcal{D}$, we know that the policy $\pi^{\mathcal{B}}$ must have preempted that request sometime along the way before time t, i.e., $d_k < t$. From the fact that request k is being kept by OPT until time t we conclude that request k had not reached its deadline at time d_k, meaning that the $\pi^{\mathcal{B}}$ had intentionally preempted request k at time d_k. This means that the indices of all requests that $\pi^{\mathcal{B}}$ was going to serve at time d_k must have been all at least as high as the index of request k at that time d_k; in other words,

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(d_k)}\{\mathcal{B}_{r_{z_i}}(d_k)\} \ge \mathcal{B}_{r_{z_k}}(d_k).$$

Next we will use the claim, where you replace t, t' in the claim by d_k, t respectively. Now from the fact that the request k must have been arrived as late as its departure time by $\pi^{\mathcal{B}}$ and its deadline must have been as early as its departure time by OPT, i.e., $t_k \le d_k \le t \le t+1$, we conclude that $t\in[d_k, d_k+1)$. Combining the above with (D.1.2), we see that the assumption in the claim holds. Therefore,

$$\min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\, j\in\mathcal{H}_{\tilde{\pi}}(t)\right\}}\{\mathcal{B}_{r_{z_i}}(t)\} \ge \mathcal{B}_{r_{z_k}}(t).$$

Finally, notice that we have showed the above inequality $\forall k\in\mathcal{D}$. Moreover, notice that $|\mathcal{H}_{\mathsf{OPT}}(t)|\le C$ as well as $\mathcal{J}\subseteq\mathcal{H}_{\pi^{\mathcal{B}}}(t)\subseteq\mathcal{H}_{\tilde{\pi}}(t)$. It follows that

$$\mathcal{H}_{\mathsf{OPT}}(t)\preceq\mathcal{H}_{\tilde{\pi}}(t) \implies \sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)}\mathcal{B}_{r_{z_i}}(t) \ge \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)}\mathcal{B}_{r_{z_i}}(t).$$

The proof of Lemma 3 is completed.

□

D.1.3. Proof of Lemma 4

Proof of Lemma 4. The argument is similar to the one of Lemma 2. First, notice that we have: $\mathsf{OPT}(\mathcal{I}) = \sum_{i\in[N]} h_i(d_i^* - t_i)$, $V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right)$. Hence, it is sufficient to show that for all $i \in [N]$, we have

$$\left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_o h_i(d_i^* - t_i).$$

This inequality itself is again a direct consequence of the definition of γ_o. If we set $z = z_i$ and $t' = d_i^* \in [t_i, t_i + 1)$, then it follows immediately from the definition of γ_o in (7):

$$\frac{\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt}{h_i(d_i^* - t_i)} \geq \gamma_o,$$

completing the proof.

□

D.1.4. Proof of Theorem 1

Proof of Theorem 1. To prove Theorem 1, we combine the result of all three Lemmas 2 to 4 together, and we can get what we want:

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_a \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_a \gamma_o \mathsf{OPT}(\mathcal{I}).$$

□

D.2. Proofs in Subsection 4.3

D.2.1. Proof of Lemma 5

Proof of Lemma 5. Please refer to Section 7 and Lemma 7 for the derivation of γ_a and γ_o under a more generalized setting, in which the reward rate of each request can vary over time. This setting is a special case of that when $R = 1$ (refer to there for definition of $\mathfrak{r}(.)$ and R), and the height of each request is a constant function (meaning $\mathfrak{r}(.) = 1$) which clearly satisfies both homogeneity and non-increasing assumptions needed in Lemma 7. □

D.2.2. Proof of Theorem 2

Proof of Theorem 2. The lower bound part, i.e., $\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) \geq \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68$ can be readily achieved by combining Theorem 1 and Lemma 5. The rest is to show that this ratio is, indeed, tight. We prove this by providing a bad instance $\mathcal{I}$ under which the performance ratio of $\pi^{\mathsf{exp}(\alpha^*)}(\mathcal{I})$ against $\mathsf{OPT}(\mathcal{I})$ is, in fact, equal to $\frac{1-e^{-\alpha^*}}{\alpha^*}$. Instance $\mathcal{I}$ consists of only 2 type of requests each duplicated C times, i.e., $z_1 := (t_1 = 0, h_1 = 1)$ and $z_2 := (t_2 = 1^-, h_2 = e^{-\alpha^*})$ where we have C duplicate of each of them. Under this instance, the policy will preempt all of the second type requests immediately and just keep serving all first types till their deadline, $t = 1$. However, the optimal offline policy will preempt all the first type requests when the second type ones arrive and start serving them until the end of their deadline which is $t = 2^-$. As a result we will have $U_{\pi^{\mathsf{exp}(\alpha^*)}}(\mathcal{I}) = C$, while $\mathsf{OPT}(\mathcal{I}) = C + Ce^{-\alpha^*} = C(1 + e^{-\alpha^*})$. Therefore, their ratio will be:

$$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) \leq \frac{U_{\pi^{\mathsf{exp}(\alpha^*)}}(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})} = \frac{1}{1 + e^{-\alpha^*}} = \frac{1 - e^{-\alpha^*}}{\alpha^*}.$$

Note that the last equality comes from the choice of α^*, i.e., $1 - \alpha^* = e^{-2\alpha^*} \to \alpha^* = 1 - e^{-2\alpha^*} = (1 - e^{-\alpha^*})(1 + e^{-\alpha^*})$. This concludes the tightness of the competitive ratio of this policy, i.e., $\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) = \frac{1-e^{-\alpha^*}}{\alpha^*}$.

□

Appendix E: Proofs in Section 5

E.1. Proof of Proposition 3

Proof of Proposition 3. Consider the continuum of arrival of requests over the time interval $[0,1]$, where at each time t we input C duplicates of request $z_t := (t, h_t := 1-t), \forall t \in [0,1]$. It is not hard to see that an optimal policy is to serve all the C duplicates of the request z_0 until the end of their lifespan, i.e., $t=1$. Therefore, we get $\mathsf{OPT} = C$. On the other hand, $\mathsf{Skyline}$ not only will keep serving all C duplicates of the z_0 until $t=1$, but also it will serve all the departed requests for an infinitesimal duration of time at their deadline. As a result, we would have $\mathsf{Skyline} = C(1+\int_1^2 h_{t-1}dt) = C\frac{3}{2}$. This will tell us that there is at least a gap of $\frac{2}{3}$ on the ratio of OPT over $\mathsf{Skyline}$ under worst-case instance, so as on the ratio of any policy π against the $\mathsf{Skyline}$. □

E.2. Proof of Proposition 4

Proof of Proposition 4. We prove this proposition by providing a strategy for the adversary that any feasible deterministic online algorithm will perform poorly under that. In particular, given any feasible deterministic online algorithm π, adversarial instance $\mathcal{I}^{Adv}$ is constructed in the following manner. Suppose the first request arrives with $z_1 := (t_1 = 0, h_1 = 1)$. After that, the second request will arrive with parameters $z_2 := (t = x^*, h = x^*)$, where $x^* \approx 0.597$ is the unique solution to $\frac{1}{2x} = \frac{x(2-x)}{1}$ on $[0,1]$. At time x^* any algorithm needs to decide which one of the request 1 or 2 should be preempted from the system. If π decided to preempt request 2 and continue on serving request 1, the adversary will cut any further arrival to the system, i.e., make request 2 the last arrived request. On the other hand, if the policy decided to preempt request 1 and start serving request 2, then at time $t=1$, the adversary will restart the instance and repeat this process for a new unit time, i.e., it assumes $t=0$, and it will input to the system the two requests 1 and 2 as specified earlier.

Now we claim that under instance $\mathcal{I}^{Adv}$ - no matter how many rounds are passed based on the response of the algorithm, where in each round k the adversary inputs a pair of requests (request $z_{2k-1} := (k-1, 1)$ and request $z_{2k} := (k-1+x^*, x^*)$) - the ratio of the total reward that π has accumulated compared to that of the optimal offline would be equal to $x^*(2-x^*)$.

To prove this claim, suppose a total of K rounds have passed until the instance $\mathcal{I}^{Adv}$ got terminated by the adversary. This means that the algorithm has accepted to serve all new arrivals until the arrival of the next request, except for the last request $2K$ for which the algorithm has chosen to preempt it immediately and continued serving the request $2K-1$ until its deadline $t_{2K-1}+1 = K$.

As for the optimal offline, it will only serve the odd requests and preempts all even requests immediately, except the last request $2K$. More specifically, all requests $2i-1$ for all $i \in [K-1]$ will be served in the system for a whole unit of time, request $2K-1$ will be served for period with length x^*, and lastly, request $2K$ will be served for a whole unit of time again.

Accordingly, the utility that algorithm accumulates in each round $i \in [K-1]$ (the time interval $[i-1, i)$) is equal to $x^*(2-x^*)$, while it accumulated a total reward of 1 from the request $2K-1$, and nothing more. On the other hand, the optimal offline is accumulating a total reward of 1 in each of the rounds $i \in [K-1]$, as well as a total of x^* from each of the requests $2K-1$ and $2K$.

Finally, we get that the ratio of the reward of the algorithm to optimal offline in each of the rounds $i \in [K-1]$ is $\frac{x^*(2-x^*)}{1}$, while the ratio of their total reward after time $t = K-1$ would be $\frac{1}{2x^*}$. Now due to the fact that x^* satisfies $\frac{1}{2x^*} = \frac{x^*(2-x^*)}{1}$, we can conclude that the ratio of total reward of the algorithm compared to that of the optimal offline under such instance $\mathcal{I}^{Adv}$ is $x^*(2-x^*)$, concluding the proof of the claim. Consequently, our proposition is proved from the fact that the competitive ratio is, in fact, the minimum ratio under such adversarial instances (strategies). □

E.3. Proof of Proposition 5

Proof of Proposition 5. We will provide a mixed strategy for the oblivious adversary. Recall the strategy the adversary was using in the proof of Proposition 4 in Subsection E.2. It was a repeated round instance, where in each round k, the adversary was inputting the two requests $z_1 := (t_1 = k-1, h_1 = 1)$ and $z_2 := (t = k-1+x^*, h = x^*)$, where $x^* \approx 0.597$ was the solution to $\frac{1}{2x} = \frac{x(2-x)}{1}$. In there, the stopping time K (end of the last round K when the adversary terminates sending any new round after that) was computed based on the decisions of the policy π, which was deterministic.

Now that the policy π can be a randomized policy, we consider the instance $\mathcal{I}$ in which the adversary will input the same pair of requests in each round k - with parameters exactly as described in the deterministic version in Subsection E.2 - with probability $\frac{1}{2}$ independently at random, and with probability $\frac{1}{2}$, the adversary skips feeding the pair in round k and awaits until the time of next round $k+1$ to arrive and repeats this coin flipping process forever. Under such a random instance, it is not hard to see that at any time k (the end of round k), if the next round would be skipped then the optimal algorithm will continue serving the request it had from round k until its deadline, and if the round $k+1$ arrives, then it is optimal to switch to the fresh z_1 (type-1) request of the new round.

With this in mind, we can see that at any time $k-1+x^*$ (the time in round k that the algorithm should decide whether or not to switch from serving the type-1 request z_1 to the type-2 request z_2) no matter what decision it takes, the competitive ratio of its decision (corresponding to the total reward achieved only from the requests in round k) will be the same and equal to $\frac{1+x^*(2-x^*)}{2} < \frac{1+0.8376}{2} \approx 0.92$, and for the next non-skipped rounds everything will be reset due to the fact that we showed any optimal policy will switch at the beginning of a non-skipped round.

Accordingly, with this mixed strategy, the adversary has made any feasible online policy π to be indifferent in the middle-round decisions. Thus, showing us that the competitive ratio of any feasible randomized online policy would be now upper bounded by $\frac{1+x^*(2-x^*)}{2} < \frac{1+0.8376}{2} \approx 0.92$. □

Appendix F: Proofs of Section 6

F.1. Proofs in Subsection 6.1

F.1.1. Proof of Theorem 3

Proof of Theorem 3. We prove that the following chain of inequalities holds:

$$U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq U_{\widetilde{\pi}}(\widetilde{\mathcal{I}}) \geq \widetilde{\mathcal{CR}} \times \mathsf{OPT}(\widetilde{\mathcal{I}}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I}).$$

To do so, let us prove each of the three inequalities one by one:

1. $U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq U_{\widetilde{\pi}}(\widetilde{\mathcal{I}})$: We prove that for any request i, the total reward corresponding to it that π^{red} acquires is higher than the amount that $\widetilde{\pi}$ acquires. To see this, we consider all possible cases in the following:

(a) $\ell_i \geq \sqrt{L}$: In this case, we know $d_i^{\pi^{\mathsf{red}}} = d_i^{\widetilde{\pi}}$, and the arrival time and the reward rate for such a request were the same under both $\widetilde{\mathcal{I}}$ and $\mathcal{I}$, hence they both receive exactly the same reward from request i.

(b) $\ell_i < \sqrt{L},\ d_i^{\widetilde{\pi}} \leq t_i + \ell_i$: In this case, we again know $d_i^{\pi^{\mathsf{red}}} = d_i^{\widetilde{\pi}}$. Additionally, notice that we had $h_i > \widetilde{h_i}$; therefore, we can again get $h_i(d_i^{\pi^{\mathsf{red}}} - t_i) \geq \widetilde{h_i}(d_i^{\widetilde{\pi}} - t_i)$.

(c) $\ell_i < \sqrt{L},\ d_i^{\widetilde{\pi}} > t_i + \ell_i$: In this case, we have $d_i^{\pi^{\mathsf{red}}} = t_i + \ell_i$. As a result, we get $h_i(d_i^{\pi^{\mathsf{red}}} - t_i) = h_i\ell_i = \widetilde{h_i}\widetilde{\ell_i} \geq \widetilde{h_i}(d_i^{\widetilde{\pi}} - t_i)$, which is again what we wanted to prove.

2. $U_{\widetilde{\pi}}(\widetilde{\mathcal{I}}) \geq \widetilde{\mathcal{CR}} \times \mathsf{OPT}(\widetilde{\mathcal{I}})$: Proof of this part is just by definition, i.e., $\widetilde{\mathcal{CR}}$ was, in fact, the competitive ratio of $\widetilde{\pi}$ under the homogeneous lifespan setting, which subsumes $\widetilde{\mathcal{I}}$.

3. $\mathsf{OPT}(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$: In order to prove this last inequality, we will provide a feasible policy π under the homogeneous instance $\widetilde{\mathcal{I}}$ whose utility satisfies: $\mathsf{OPT}(\widetilde{\mathcal{I}}) \geq U_\pi(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$. Note that the first inequality automatically follows by definition of optimality of $\mathsf{OPT}(\widetilde{\mathcal{I}})$ and feasibility of π under $\widetilde{\mathcal{I}}$. To define behavior of π under $\widetilde{\mathcal{I}}$ uniquely, again we only need to determine its departure times for all request in $\widetilde{\mathcal{I}}$. Thus, we set $d_i^\pi = \min(d_i^{\mathsf{OPT}(\mathcal{I})}, t_i + \widetilde{\ell_i})$. By this way of defining the departure time as well as the fact that $\mathsf{OPT}(\mathcal{I})$ was feasible under $\mathcal{I}$ (w.r.t. capacity and lifespan constraints), we can easily see that π is feasible under $\widetilde{\mathcal{I}}$. The last remaining part to show is the inequality $U_\pi(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$. Similarly, we will show that for any request i, the total amount of reward corresponding to i under both $\pi(\widetilde{\mathcal{I}})$ and $\mathsf{OPT}(\mathcal{I})$, satisfies the inequality. Consider the following two possible cases:

(a) $\ell_i \geq \sqrt{L}$: For this class of requests, we have $\widetilde{h_i} = h_i$ and $d_i^\pi - t_i = \min(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i, \sqrt{L}) \geq \frac{1}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, because $d_i^{\mathsf{OPT}(\mathcal{I})} - t_i \leq L$. As a consequence, we get $\widetilde{h_i}(d_i^\pi - t_i) \geq h_i \frac{1}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, which is exactly what we wanted to show.

(b) $\ell_i < \sqrt{L}$: For this other class of requests, we have $d_i^\pi = d_i^{\mathsf{OPT}(\mathcal{I})}$ and $\widetilde{h_i} = \frac{h_i\ell_i}{\sqrt{L}} \geq \frac{h_i}{\sqrt{L}}$, because $\ell_i \geq 1$. As a result, we get $\widetilde{h_i}(d_i^\pi - t_i) \geq \frac{h_i}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, which is again what we wanted to show.

Putting all these three pieces of chain inequalities together, we conclude the proof of the theorem.

□

F.1.2. Proof of Proposition 6

Proof of Proposition 6. It is enough to provide an adversary instance generation strategy under which no deterministic feasible online algorithm is able to beat $\frac{1}{\sqrt{L}}$ fraction of the OPT utility. The bad instance $\mathcal{I}$ will be generated as follows. Fix a deterministic feasible online algorithm π. At time $t = 0$, the adversary will simultaneously input two requests $z_1 = (t_1 = 0, h_1 = \sqrt{L}, \ell_1 = 1)$ and $z_2 = (t_2 = 0, h_2 = 1, \ell_2 = L)$. If π chooses to serve z_1, then the adversary will terminate the instance, i.e., will send no more arrivals to the system. Otherwise, i.e., if π chooses to serve z_2, then at time $t = 1$, the adversary imagines $t = 0$ and repeat the entire pair arrival again, with two new requests z_3 and z_4 that are defined in exactly the same way as above, except for their arrival time which is now 1 for both of them. The adversary will continue on inputting these pairs of requests (at integer times) until the first time, say at round k, that the algorithm decides to serve the request z_{2k-1} and preempt request z_{2k}, after which the adversary will send no more arrivals to the system.

To see that why the utility ratio of the algorithm against optimal offline under such an instance is at most $\frac{1}{\sqrt{L}}$, suppose the instance consists of a total of K repeated rounds before it gets terminated. The algorithm must have chosen to serve all even requests $z_{2k}, \forall k \in [K-1]$ as well as request z_{2K-1}, while the optimal offline - under such instance - will choose to serve all odd requests $z_{2k-1}, \forall k \in [K-1]$ together with the last request z_{2K}. Thus, the utility of the algorithm $U_\pi(\mathcal{I}) = (K-1) + \sqrt{L}$, whereas that of the optimal offline would be $\mathsf{OPT}(\mathcal{I}) = (K-1)\sqrt{L} + L$, which generates a competitive ratio of at most $\mathcal{CR}(\pi) \leq \frac{1}{\sqrt{L}}$ and concludes our proof. □

F.2. Proofs in Subsection 6.2

F.2.1. Proof of Lemma 6

Proof of Lemma 6. The proof of the lemma concludes from the following chain of inequalities:

$$\begin{aligned} U_{\pi_K^{\mathsf{div}}}(\mathcal{I}; C) &= \sum_{k\in[K]} U_{\pi^k}\left(\mathcal{I}_k; \lfloor \frac{C}{K} \rfloor\right) \\ &\geq \sum_{k\in[K]} \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \mathsf{OPT}\left(\mathcal{I}_k; \lfloor \frac{C}{K} \rfloor\right) \\ &\geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \sum_{k\in[K]} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}_k; C) \\ &\geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}; C). \end{aligned}$$

Before delving into the rationale behind each inequality, it is essential to recall that the second argument (C or $\lfloor \frac{C}{K} \rfloor$) in the utility functions ($U_\pi(\mathcal{I}; C)$ or $\mathsf{OPT}(\mathcal{I}; C)$) represents the system capacity under which the algorithm operates. For the sake of brevity, we had omitted this parameter so far, as our analysis focused solely on a single value of capacity. However, with the introduction of distinct capacities for our sub-problems compared to the original problem, it becomes necessary to reintroduce this parameter for clarity. Now let us state the reasoning behind each step of this chain of inequalities:

1. The first equality holds because π_K^{div} is exactly emulating all of the π^k, $\forall k \in [K]$, simultaneously.

2. The second inequality holds from the fact that the competitive ratio of each π^k (with capacity $\lfloor \frac{C}{K} \rfloor$) under $\mathcal{I}_k$ was $\frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}}$.

3. The third inequality holds from the simple observation that if you multiply the capacity of OPT by some factor, its utility will be increased by at most that factor.

4. The fourth inequality results from the following two facts. First, $\mathcal{I} = \cup_{k\in[K]} \mathcal{I}_k$. Second, for any $k \in [K]$, the way $\mathsf{OPT}(\mathcal{I}; C)$ serves just the requests in $\mathcal{I}_k$, would itself be a feasible policy under the instance $\mathcal{I}_k$ when capacity is still C. Therefore, the share of the utility $\mathsf{OPT}(\mathcal{I}; C)$ that comes from the requests in $\mathcal{I}_k$ is less than the $\mathsf{OPT}(\mathcal{I}_k; C)$. Finally, by taking the summation over all categories, we get the inequality we want, since from the first fact we are sure that we have accounted for each request's share in $\mathsf{OPT}(\mathcal{I}_k; C)$ at least once.

□

F.2.2. Proof of Theorem 4

Proof of Theorem 4. From Lemma 6, we know that

$$\mathcal{CR}\left(\pi^{\mathsf{div}}_{K^*}\right) \geq \max_{K\in[C]} \left\{ \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \right\}.$$

Hence, it is sufficient to show that whenever $C = \Omega(\log(L))$, we have

$$\max_{K\in[C]} \left\{ \frac{1}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \right\} = \Omega\left(\frac{1}{\log(L)}\right).$$

Let w be a small constant such that $C \geq w\log(L)$. Now let $v \ll w$ also be a constant, which is much smaller than w, say $\frac{w}{v} > 11$. Now let $\hat{K} := \lceil v\log(L)\rceil < \frac{w\log(L)}{10} \leq \frac{C}{10}$, where L is sufficiently large ($v\log(L) > 10$). As a consequence, we will get

$$\frac{1}{\sqrt{L^{\frac{1}{K}}}} \geq \frac{1}{e^{\frac{1}{2v}}} = \Omega(1), \quad \frac{\lfloor \frac{C}{\hat{K}} \rfloor}{C} \geq \frac{\frac{C}{\hat{K}} - 1}{C} = \left(\frac{1}{\hat{K}} - \frac{1}{C}\right) = \frac{0.9}{\hat{K}} + \left(\frac{1}{10\hat{K}} - \frac{1}{C}\right) \geq \frac{0.9}{\hat{K}} = \Omega\left(\frac{1}{\log(L)}\right).$$

Thus, we get $\max_{K\in[C]} \left\{ \frac{1}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \right\} \geq \frac{1}{\sqrt{L^{\frac{1}{\hat{K}}}}} \frac{\lfloor \frac{C}{\hat{K}} \rfloor}{C} = \Omega\left(\frac{1}{\log(L)}\right)$, completing the proof of the theorem. □

Appendix G: Proofs in Section 7

G.1. Proof of Lemma 7

Proof of Lemma 7. The two equalities are simply from assumption (11) and how we define r_z, which equals $\mathfrak{h}(t)e^{\alpha t}$. The two inequalities can be derived by setting $t' \uparrow t+1$ and $t' \downarrow t$, respectively. Lastly, if the function $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ is non-increasing in t'' on $[0,1]$, then we claim that

$$\frac{\int_0^{t'-t} e^{-\alpha t''}\ dt''}{\int_0^{t'-t} \mathfrak{r}(t'')\ dt''} \tag{EC.6}$$

is also non-increasing in t' on $[t, t+1]$. This is due to the fact that if a ratio of two functions $\frac{a(t)}{b(t)}$ is non-increasing in t and $b(t) > 0$ for all $t \geq 0$, then $\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}$ would also be non-increasing in T for all $T \geq 0$. To see this, note that

$$\frac{d\left(\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right)}{dT} = \frac{a(T)\int_0^T b(t)dt - b(T)\int_0^T a(t)dt}{\left(\int_0^T b(t)dt\right)^2} = \frac{b(T)}{\int_0^T b(t)dt}\left[\frac{a(T)}{b(T)} - \frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right],$$

where

$$\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt} = \frac{\int_0^T b(t)\frac{a(t)}{b(t)}dt}{\int_0^T b(t)dt} \geq \frac{\int_0^T b(t)\frac{a(T)}{b(T)}dt}{\int_0^T b(t)dt} = \frac{a(T)}{b(T)},$$

which implies $\frac{d\left(\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right)}{dT} \leq 0$. The monotonicity of (EC.6) implies that the minimum value for each of γ_o and γ_a's min operator is, in fact, achieved when $t' \uparrow t+1$ and $t' \downarrow t$, respectively, thereby making the inequalities hold with equality and completing the proof of the lemma. □